UNIVERSITÀ DI TRENTO

Department of Information Engineering and Computer Science

Bachelor's Degree in
Computer, Communications and Electronic Engineering

Final Dissertation

Cryptography Bill of Materials

Inventory, Analysis and Risk Assessment of Cryptographic Components in Software

Supervisor
Prof. Silvio Ranise

Co-supervisors
Alessandro Tomasi
Pietro De Matteis
Luca Piras
Luis Augusto Dias Knob

Student
Claudio Foroncelli

Academic year 2024/25

Contents

Abstract **3**

1 Introduction **4**
1.1 Context and Motivation . . . 4
1.2 The Cryptography Bill of Materials . . . 4
1.3 From Inventory to Compliance . . . 4
1.4 Research Contributions . . . 5
1.5 Thesis Structure . . . 5

2 Background **6**
2.1 Introduction . . . 6
2.2 Relevant Guidance on Cryptographic Inventories and CBOMs . . . 6
2.2.1 OMB Memorandum M-23-02: Prioritized Cryptographic Inventories . . . 6
2.2.2 NIST IR 8547: Cryptographic Asset Management and Transition Planning . . . 6
2.2.3 NIST SP 1800-38B: Cryptographic Discovery and CBOM . . . 7
2.2.4 Alignment and Implications . . . 7
2.3 Standards for Cryptography Bills of Materials (CBOMs) . . . 7
2.3.1 CycloneDX . . . 8
2.3.2 SPDX . . . 8
2.3.3 FIPS 140-3 and ISO/IEC 19790 . . . 8
2.3.4 Common Criteria (ISO/IEC 15408) . . . 8
2.4 The CycloneDX CBOM Structure . . . 8
2.4.1 Overview . . . 8
2.4.2 Cryptographic Asset Types in CBOM . . . 8
2.4.3 Serialization Examples . . . 11
2.5 Tools for Generating CBOMs . . . 11
2.5.1 IBM CBOMkit . . . 11
2.5.2 CycloneDX cdxgen . . . 13
2.5.3 GitHub CodeQL CBOM Action . . . 13
2.5.4 Rationale for Choosing CBOMkit . . . 13

3 Analysis and compliance **14**
3.1 Introduction . . . 14
3.2 Background on Policies and Rules . . . 14
3.2.1 TOML as the Configuration Language for CBOMkit Policies . . . 15
3.3 Policy structure . . . 15
3.3.1 Policy Header . . . 16
3.3.2 Assessment Levels . . . 16
3.3.3 Compliance Levels . . . 17
3.3.4 Rules . . . 18
3.3.5 Rule Matching and Precedence . . . 19
3.4 Implementation of the Policy Evaluation Engine . . . 19
3.4.1 The Compliance Service Selector . . . 20

3.4.2 The Custom Policy Parser . . . 21
3.4.3 The Custom Compliance Service . . . 22

4 Results **24**
4.1 Introduction . . . 24
4.2 Policy Basis . . . 24
4.3 Evaluation of the Test CBOM . . . 25
4.4 Evaluation of the Keycloak CBOM . . . 27
4.5 Discussion . . . 29

5 Conclusion and Future Work **30**
5.1 Summary of Contributions . . . 30
5.2 Future Work . . . 30

Bibliography **31**

A Code Listings **33**
A.1 Compliance Service Selector . . . 33
A.2 Custom Policy Parser . . . 34
A.3 Custom Compliance Service . . . 37

B Policy and CBOM Examples **42**
B.1 Policy Rules (TOML) . . . 42
B.2 Test CBOM Example (JSON) . . . 43
B.3 Keycloak CBOM Excerpt (JSON) . . . 44

Abstract

This thesis explores the role of Cryptography Bills of Materials (CBOMs) as a foundation for systematic cryptographic risk management, compliance verification, and preparation for the post-quantum cryptography (PQC) transition. A CBOM is a structured, machine-readable inventory that enumerates the algorithms, keys, certificates, and protocols used in a software system, providing the visibility needed to detect deprecated components and ensure alignment with regulatory guidelines.

The research began with a survey of CBOM standards and related tools, identifying the software CBOMkit, originally developed by IBM, as the most complete open-source solution for cryptographic inventories. Building on this foundation, the thesis introduces a policy-driven evaluation engine, implemented in Java, capable of assessing CBOMs against customizable compliance policies expressed in a TOML format. The engine defines compliance and assessment levels to classify individual assets and aggregate them into an overall system posture.

The extended framework is validated through experiments on a test CBOM and on the CBOM generated from Keycloak, a widely used open-source identity and access management platform. The results show that the system reliably identifies deprecated and disallowed primitives such as SHA-1, 3DES, and AES in ECB mode, and communicates these findings through compliance reports.

These contributions demonstrate that CBOM inventories, coupled with automated evaluation, are important for compliance, security, and PQC migration. In particular, they highlight a key principle: organizations cannot effectively replace cryptography for the quantum era unless they first know precisely what is already in use.

1 Introduction

1.1 Context and Motivation

Cryptography is a fundamental building block of secure software systems. It underlies essential mechanisms such as confidentiality, integrity, authentication, and non-repudiation. As modern software ecosystems become increasingly complex, applications rely on a wide variety of cryptographic primitives, protocols, and key management techniques. Ensuring that these components are correctly implemented and conform to regulatory standards is a central challenge in cybersecurity.

At the same time, the regulatory landscape is becoming more demanding. Agencies such as NIST and ENISA [19, 21, 20, 5, 24, 1] publish recommendations and standards that define which algorithms, key lengths, and modes of operation are considered secure, deprecated, or disallowed. With the advent of quantum computing, the urgency to transition towards post-quantum cryptography (PQC) adds further complexity. In this environment, organizations must not only identify which cryptographic components they are using, but also assess whether these components comply with established guidelines. Accurate visibility of cryptographic assets is therefore a prerequisite for both regulatory compliance and long-term cryptographic agility.

1.2 The Cryptography Bill of Materials

The concept of a Cryptography Bill of Materials (CBOM) [22, 9] has recently emerged as a means to provide such visibility. Similar to a Software Bill of Materials (SBOM), which enumerates software components and dependencies, a CBOM is a structured, machine-readable inventory that captures the cryptographic algorithms, keys, protocols, and certificates present in a system.

By explicitly cataloguing these elements, CBOMs enable organizations to answer key questions such as:

- Which cryptographic primitives are in use, and where?
- Are any deprecated or disallowed algorithms still present?
- Do keys and parameters meet the required security levels?
- Which components will need replacement during the post-quantum cryptography transition?

The CBOM thus bridges a critical gap between high-level compliance standards and practical security management. It serves as both an authoritative record for audits and a foundation for automated compliance verification. Although the concept is still evolving, initiatives from NIST and OWASP (e.g., CycloneDX) [22, 9] are actively shaping standardized formats, while several software solutions have begun to make CBOM generation and analysis feasible in practice.

1.3 From Inventory to Compliance

Once a Cryptography Bill of Materials is generated, the cryptographic components, or assets, it captures can be used to analyze and verify the conformity of a program to compliance guidelines.

These guidelines may originate from official bodies [21, 24, 1] or be established internally by an organization. They define the standards for the safe and secure use of cryptographic components,

not only in the context of post-quantum cryptography but also with respect to outdated or legacy algorithms and contemporary cryptographic practices.

As threats evolve, for example, with the emergence of quantum computers, and regulatory requirements become more complex, it is increasingly important to have a flexible and extensible system that can express and enforce compliance policies in a precise and adaptable way.

1.4 Research Contributions

The primary goal of this thesis is to extend the CBOM concept beyond inventory generation, enabling systematic compliance evaluation of cryptographic assets.

While the work was guided by high-level research objectives, the main value of the thesis lies in its concrete contributions, which can be summarized as follows:

- Investigate the literature and standards that motivate CBOMs, and survey existing tools for generating and analyzing them.
- Identify and select the most appropriate existing CBOM tool to serve as the foundation for implementation, with the aim of applying and extending it for compliance analysis.
- Design a structured, human-readable policy format capable of expressing compliance rules and classification levels.
- Implement a CBOM evaluation engine in Java, designed to be flexible and adaptable to different compliance policies and guidelines.
- Validate the framework through experiments on both a test CBOM and a real-world CBOM derived from the open source software Keycloak [16].

Together, these contributions demonstrate that cryptographic compliance can be operationalized in a structured, automatable way, producing results that are both accurate at the asset level and meaningful at the system level.

1.5 Thesis Structure

The remainder of this thesis is organized as follows:

- Chapter 2 introduces the concept of Cryptographic Bills of Materials (CBOMs). It reviews the literature and standards that motivate their adoption, explains why they are needed, and discusses existing structures for representing cryptographic inventories, with particular focus on the CycloneDX CBOM format.
- Chapter 3 describes the extension of CBOMkit, an open-source tool developed by IBM, through the addition of a custom policy evaluation engine. It presents the design of a structured policy format and shows how this engine interprets and applies compliance rules to cryptographic assets, enabling systematic evaluation of CBOMs against different security guidelines.
- Chapter 4 reports the experimental evaluation, first using a synthetic test CBOM and then a real-world CBOM generated from Keycloak. It highlights the ability of the framework to detect deprecated and disallowed cryptographic primitives.
- Chapter 5 concludes the thesis by summarizing the main contributions and outlining directions for future research.

2 Background

2.1 Introduction

A Cryptography Bill of Materials (CBOM) is a structured, machine-readable inventory that enumerates all cryptographic components present in a software system. Similar to a Software Bill of Materials (SBOM), which catalogues software dependencies and libraries, a CBOM focuses specifically on cryptographic assets such as algorithms, keys, certificates, and protocols. By providing an authoritative record of these elements, the CBOM becomes a valuable resource for security audits, compliance verification, and cryptographic risk assessments.

The concept adapts the general "bill of materials" approach to the unique requirements of cryptographic management. It offers visibility into the cryptographic posture of a system, enabling detection of deprecated algorithms, weak key lengths, or misconfigured protocols. At the same time, it provides input for compliance checks against guidelines and standards from bodies such as NIST, ENISA, and national cybersecurity agencies.

Standardization is crucial: by representing cryptographic information in a uniform format, CBOMs enable automation, interoperability, and traceability across tools and processes. This is especially important in contexts such as post-quantum cryptography (PQC) migration, where timely identification of vulnerable assets is essential for planning secure transitions.

2.2 Relevant Guidance on Cryptographic Inventories and CBOMs

Two recent U.S. government publications provide context for the CBOM concept and its role in PQC migration.

2.2.1 OMB Memorandum M-23-02: Prioritized Cryptographic Inventories

The Office of Management and Budget (OMB) Memorandum M-23-02 [25] implements requirements from National Security Memorandum 10, mandating that all Federal agencies maintain a *prioritized inventory of cryptographic systems* as preparation for PQC migration.

The memorandum specifies that, by May 4, 2023 and annually until 2035, agencies must identify all active cryptographic systems that:

- Operate on high-impact systems or High Value Assets (HVAs), or
- Are likely to be vulnerable to attacks by a cryptanalytically relevant quantum computer.

For each system, agencies must record details such as algorithms in use, cryptographic function, key length, implementation type, operating environment, hosting details, and data lifecycle or retention requirements. Appendix B lists quantum-vulnerable algorithms (RSA, ECC, DH, DSA), closely matching those flagged in NIST SP 1800-38B [21].

Although M-23-02 does not use the term "CBOM," its inventory requirements align closely with the concept.

2.2.2 NIST IR 8547: Cryptographic Asset Management and Transition Planning

NIST Interagency Report 8547 (Initial Public Draft) [19] provides strategic guidance for the migration from quantum-vulnerable algorithms to post-quantum cryptographic standards. The report establishes

deprecation and disallowance timelines for widely used public-key algorithms such as RSA and ECC, and sets milestones for adopting quantum-resistant alternatives in federal systems.

The document highlights the importance of mapping current cryptographic usage as a prerequisite for transition. It notes that provisional measures, such as hybrid schemes combining classical and post-quantum algorithms, may be necessary to ensure interoperability during the migration period. Agencies are therefore expected to identify where vulnerable algorithms are deployed, assess their criticality, and plan phased replacements.

Although IR 8547 does not define a specific inventory mechanism, its roadmap implicitly depends on accurate and up-to-date cryptographic inventories. CBOMs can provide this structured, machine-readable view of deployed algorithms and key material, enabling organizations to track vulnerable assets and verify progress against NIST's transition milestones.

2.2.3 NIST SP 1800-38B: Cryptographic Discovery and CBOM

NIST Special Publication 1800-38B [21] describes practical methods for identifying and cataloging quantum-vulnerable cryptographic assets across enterprises. Automated *cryptographic discovery tools* are introduced to scan source code, executable files, non-executable artifacts such as keystores and certificates, and network protocols (e.g., TLS, SSH, IPsec) to produce structured inventories.

A central idea is the CBOM, defined as an extension of SBOMs with a focus on cryptographic elements. CBOMs capture algorithms, keys, protocols, and cryptographic libraries together with their relationships and dependencies. They are intended to be machine-readable, object-based records integrated into development and asset management workflows.

Within this architecture, CBOMs serve two roles: first, as comprehensive inventories that support compliance and governance, and second, as a foundation for migration planning by pinpointing instances of quantum-vulnerable algorithms such as RSA, ECDSA, ECDH, DH, and DSA. While the model is still evolving, NIST positions CBOM as a key enabler for enterprise cryptographic risk management.

On the Origin of the CBOM Format The CBOM described in NIST SP 1800-38B is not a new standard but rather a profile of *CycloneDX* [22], an SBOM specification maintained by the OWASP Foundation. CycloneDX was originally designed for software component inventories, but its extensibility allows domain-specific extensions. NIST leverages this by introducing a CBOM profile with cryptography-specific component types (e.g., `cryptographic-asset`) and properties (e.g., `cryptoProperties`, `protocolProperties`, `tlsCipherSuites`). This aligns cryptographic asset inventories with a widely supported standard while tailoring them to PQC migration needs.

2.2.4 Alignment and Implications

In summary, both policy directives and technical frameworks highlight the same key point: successful migration to post-quantum cryptography starts with knowing what cryptographic assets are currently in use. Regulations create the requirement for accurate inventories, while technical models explain how to build them. The CBOM bridges these two views by offering a standardized format that connects compliance needs with practical discovery and management. This makes it both a compliance tool and a basis for long-term cryptographic agility.

2.3 Standards for Cryptography Bills of Materials (CBOMs)

At present, only a limited number of standards directly address the structure of CBOMs. Alongside these standards, complementary assurance frameworks can be integrated into them to provide additional trust and certification metadata.

2.3.1 CycloneDX

CycloneDX, developed by OWASP, is currently the most advanced CBOM format. Initially an SBOM standard, it gained native CBOM support in version 1.6 with a dedicated `cryptographic-asset` component type and properties for algorithms, protocols, keys, and certificates [22]. Because CBOM is integrated into CycloneDX's object model, existing SBOM tooling ecosystems can be reused, making CycloneDX the *de facto* CBOM standard.

2.3.2 SPDX

SPDX (Software Package Data Exchange), developed under the Linux Foundation and standardized as ISO/IEC 5962:2021 [14], captures rich metadata on software, including licensing, copyright, and security references. While it does not yet define a CBOM profile, its extensibility means it could be adapted in the future. In contexts where ISO/IEC certification carries weight, SPDX metadata could be combined with cryptographic inventories, for example by embedding SPDX identifiers inside CBOM entries to improve traceability and compliance reporting.

2.3.3 FIPS 140-3 and ISO/IEC 19790

FIPS 140-3 [18] and ISO/IEC 19790 [13] specify internationally recognized security requirements for cryptographic modules, defining assurance levels (e.g., FIPS levels 1–4). While these standards do not prescribe inventory formats, CBOMs can incorporate such certification metadata directly. For instance, a CBOM entry describing an AES implementation could include a field noting that the library is validated under FIPS 140-3 Level 2, enabling automated compliance checks against organizational or regulatory requirements.

2.3.4 Common Criteria (ISO/IEC 15408)

The Common Criteria framework, standardized as ISO/IEC 15408 [2], defines Evaluation Assurance Levels (EAL1–EAL7) for security products. As with FIPS and ISO/IEC 19790, it does not prescribe inventory structures, but its assurance data can be embedded into CBOM entries. For example, a CBOM record for a hardware security module (HSM) could reference its Common Criteria EAL 4+ certification, allowing governance tools to evaluate not only the cryptographic functions but also the trustworthiness of their implementations.

2.4 The CycloneDX CBOM Structure

2.4.1 Overview

CycloneDX is an open standard for the software supply chain, developed and maintained by the OWASP Foundation. It was originally created for SBOMs but is extensible to other domains. From version 1.6 onward, CycloneDX natively supports the CBOM object model, enabling standardized representation of cryptographic assets and their dependencies. The CBOM profile builds on the existing CycloneDX schema, introducing a new `cryptographic-asset` component type and a set of cryptography-specific properties.

2.4.2 Cryptographic Asset Types in CBOM

The CycloneDX CBOM schema organizes cryptographic information into four main categories of assets: algorithms, certificates, protocols and related cryptographic material (such as keys, tokens, or derived values). Each category captures different aspects of cryptographic usage, from primitives and credentials to entire communication schemes. To describe these assets in a precise and machine-readable way, the schema defines a set of parameters, which are presented below.

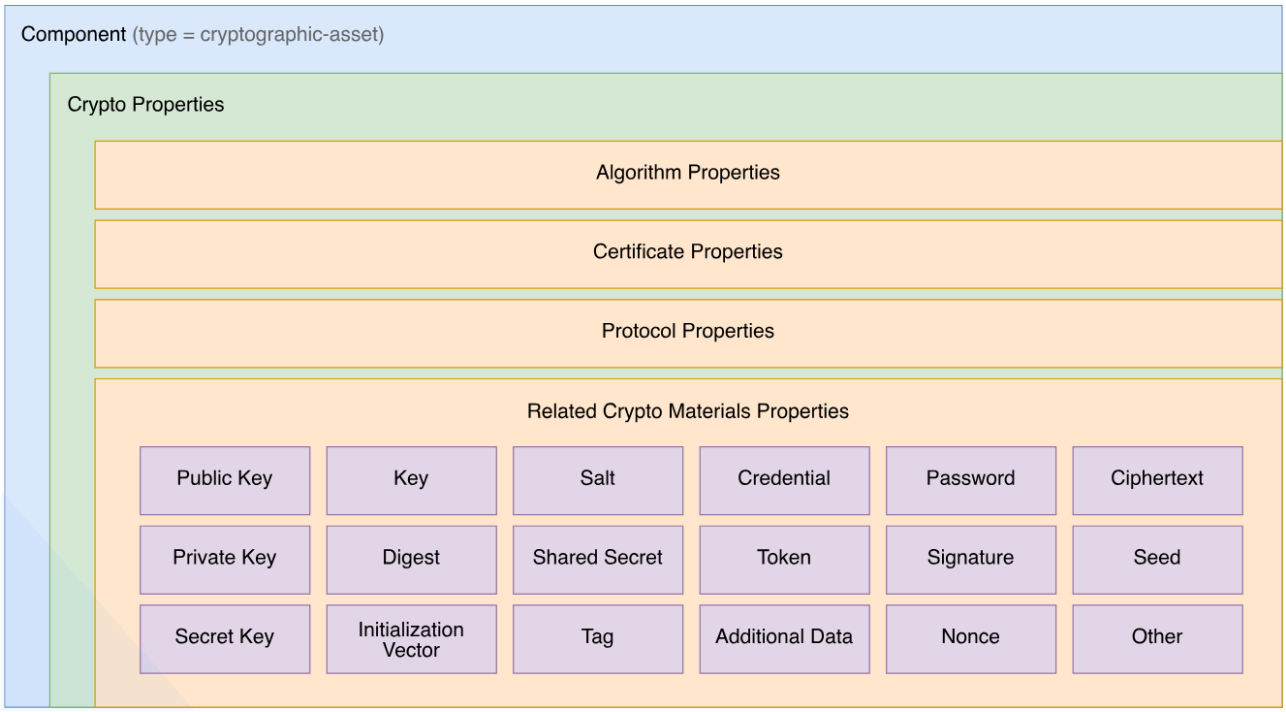

Figure 2.1: High-level view of a CycloneDX CBOM `cryptographic-asset` component: common `cryptoProperties` with specialized substructures for algorithms, certificates, protocols, and related cryptographic materials (e.g., keys, IVs, nonces, signatures) [22].

Common fields (all cryptographic assets)

All cryptographic assets share a set of common parameters within the `cryptoProperties` object:

- `assetType`: identifies which kind of cryptographic asset is described (`algorithm`, `certificate`, `relatedCryptoMaterial`, or `protocol`).
- `classicalSecurityLevel`: indicates the estimated classical (non-quantum) security strength in bits (e.g., 128, 192, 256).
- `nistQuantumSecurityLevel`: records the NIST Post-Quantum Cryptography security category (0–5), where 0 means not applicable.
- `oid`: an object identifier string that uniquely identifies a cryptographic algorithm or parameter set, when available.
- `confidenceLevels`: expresses how confident the discovery tool is about the accuracy of each detected attribute.
- `scanner`: the name of the tool or system that detected the cryptographic asset.
- `detectionContext`: contextual information about how and where the asset was discovered (e.g., file path, line numbers, code symbols, or keyword matches).

Algorithms

The `algorithmProperties` object describes cryptographic primitives and algorithms:

- `primitive`: category of the primitive (e.g., `blockcipher`, `hash`, `signature`, `kdf`).
- `variant`: free-text description of the family or variant (e.g., RSA-2048, SHA-384, Curve25519).
- `implementationLevel`: indicates whether the algorithm is implemented in software, hardware, a secure enclave/TEE, or other.

- `implementationPlatform`: specifies the platform (e.g., x86-64, ARMv8, s390x, generic).
- `certificationLevel`: whether the implementation is certified under FIPS 140-3 (levels 1–5), Common Criteria (EAL1–EAL7), or similar.
- `mode`: operation mode for block ciphers (e.g., CBC, GCM, CTR).
- `padding`: padding scheme (e.g., PKCS#7, OAEP, raw).
- `cryptoFunctions`: operations provided (e.g., `encrypt`, `decrypt`, `sign`, `verify`, `keygen`).

Certificates

The `certificateProperties` object describes certificates, typically X.509:

- `subjectName`: entity identified by the certificate.
- `issuerName`: the issuing authority or CA.
- `notValidBefore`: the start of certificate validity.
- `notValidAfter`: the expiration of certificate validity.
- `certificateAlgorithm`: algorithm of the certificate's public key (e.g., RSA, ECDSA).
- `certificateSignatureAlgorithm`: algorithm used by the issuer to sign the certificate (e.g., SHA-256 with RSA).
- `certificateFormat`: the encoding (e.g., PEM, DER).
- `certificateExtensions`: any extensions such as key usage or extended key usage.

Protocols

The `protocolProperties` object describes cryptographic protocols:

- `tlsCipherSuites`: supported TLS cipher suites (e.g., `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`).
- `ikev2TransformTypes`: specifies IKEv2 transforms, divided into:
  - `transformType1`: encryption algorithms.
  - `transformType2`: pseudorandom functions (PRFs).
  - `transformType3`: integrity algorithms.
  - `transformType4`: Diffie–Hellman groups.

Related cryptographic material

The `relatedCryptoMaterialProperties` object captures keys and related artifacts:

- `relatedCryptoMaterialType`: specifies the type of material (e.g., `privateKey`, `publicKey`, `secretKey`, `ciphertext`, `nonce`, `token`).
- `size`: size of the material in bits (e.g., 256, 2048).
- `format`: encoding or representation (e.g., PEM, DER, PKCS#8).
- `secured`: boolean flag indicating whether the material is stored or managed securely (e.g., encrypted at rest, inside an HSM).

2.4.3 Serialization Examples

CBOM inherits CycloneDX's serialization formats (JSON, XML, Protocol Buffers). Listings 2.1 and 2.2 provide JSON examples adapted from the OWASP guide.

Listing 2.1: CBOM entry for AES-128-GCM

```
{
  "type": "cryptographic-asset",
  "name": "AES-128-GCM",
  "cryptoProperties": {
    "assetType": "algorithm",
    "algorithmProperties": {
      "primitive": "ae",
      "parameterSetIdentifier": "128",
      "mode": "gcm",
      "classicalSecurityLevel": 128,
      "nistQuantumSecurityLevel": 1
    },
    "oid": "2.16.840.1.101.3.4.1.6"
  }
}
```

Listing 2.2: CBOM entry for TLSv1.2 protocol with cipher suite

```
{
  "type": "cryptographic-asset",
  "name": "TLSv1.2",
  "cryptoProperties": {
    "assetType": "protocol",
    "protocolProperties": {
      "type": "tls",
      "version": "1.2",
      "cipherSuites": [
        {
          "name": "TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384",
          "algorithms": [
            "crypto/algorithm/ecdh-curve25519",
            "crypto/algorithm/rsa-2048",
            "crypto/algorithm/aes-128-gcm",
            "crypto/algorithm/sha-384"
          ]
        }
      ]
    }
  }
}
```

2.5 Tools for Generating CBOMs

While the CBOM model is still emerging, several tools already support the generation of cryptographic inventories in machine-readable formats.

2.5.1 IBM CBOMkit

IBM Research has developed *CBOMkit*, an open-source suite of tools dedicated to generating, analyzing, and managing Cryptography Bills of Materials [9]. Unlike general-purpose SBOM tools with limited cryptographic support, CBOMkit was designed from the ground up to address the specific challenges of cryptographic asset discovery and post-quantum readiness. It provides an end-to-end pipeline: from source-code scanning, to CBOM generation, visualization, compliance checking, and

long-term storage. Outputs are fully aligned with the CycloneDX CBOM specification, ensuring interoperability with existing security and compliance workflows.

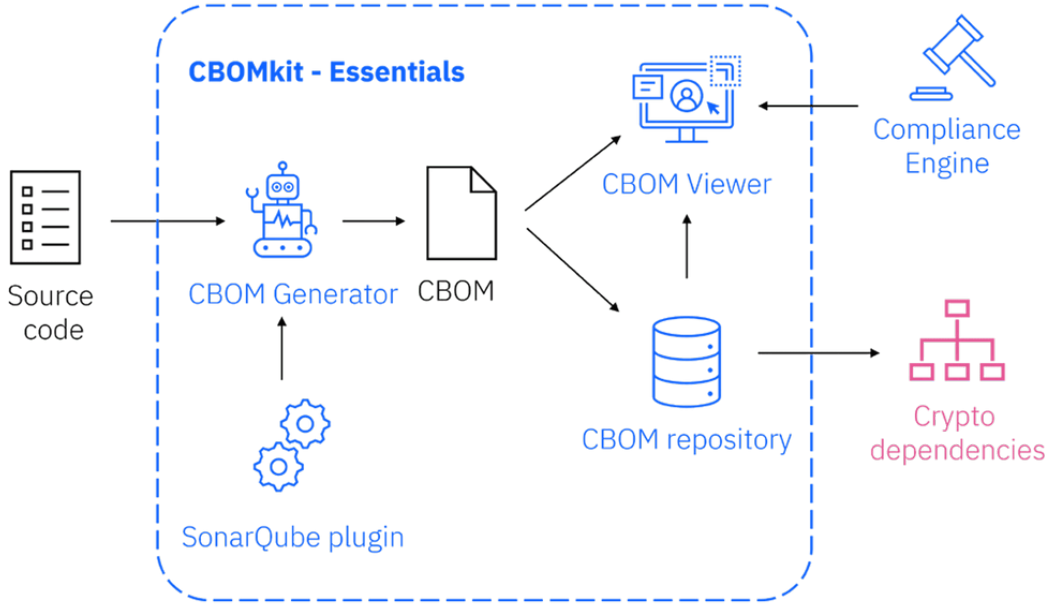

Figure 2.2: High-level architecture of IBM's CBOMkit: source code is scanned via a SonarQube plugin, generating a CBOM that can be stored, viewed, and analyzed by the compliance engine or linked to crypto dependencies [12].

The core component, CBOMkit-Hyperion, is a SonarQube plugin that performs static code analysis to identify cryptographic usage. Rather than relying on keyword or pattern matching, Hyperion employs Abstract Syntax Tree (AST) parsing to analyze API calls, parameters, and configurations with high accuracy. This approach allows it to extract detailed properties of cryptographic assets, including algorithms, modes, key lengths, padding schemes, and related metadata. At present, Hyperion supports Java (JCA and BouncyCastle) and Python (pyca/cryptography), but its modular design enables the integration of additional languages through custom detection rules.

A key strength of Hyperion lies in its rule-based detection system. Each language module defines rules that map cryptographic API calls to structured CBOM entries, specifying attributes such as algorithm variant, key size, operation mode, or padding. The detection process then enriches these entries with metadata, for example object identifiers (OIDs) or default parameter values. The rule structure is openly documented and designed for extensibility, allowing organizations to extend Hyperion to cover new APIs or libraries as needed. While current language support focuses on Java and Python, ongoing work is expanding this coverage further [10, 11].

Beyond Hyperion, CBOMkit includes a set of complementary tools that extend its functionality:

- CBOMkit-Theia: scans container images or file directories to detect cryptographic assets, complementing source-code analysis when compiled artifacts are available.
- CBOMkit-Coeus: a web-based viewer for CBOMs, enabling interactive visualization of cryptographic inventories and generating summary statistics. Coeus is particularly valuable because it provides the first dedicated interface for CBOM exploration, bridging the gap between raw JSON data and human-readable insight.
- CBOMkit-Themis: a compliance checker that evaluates CBOMs against policy definitions, for example to flag deprecated algorithms, insufficient key lengths, or non-validated modules. This makes it possible to connect inventories directly to compliance processes and PQC migration requirements.

- CBOMkit-Mnemosyne: a repository service and REST API for storing, managing, and retrieving CBOMs across projects and teams. This enables CBOMs to be integrated into CI/CD pipelines, asset management systems, or enterprise cryptographic governance workflows.

Together, these components provide a complete solution for cryptographic inventory management.

2.5.2 CycloneDX cdxgen

The OWASP CycloneDX project maintains *cdxgen* [3], a tool capable of generating SBOMs and, in recent versions, CBOMs as well [4]. It supports multiple programming languages and build systems, scanning source code and binaries to extract dependency and cryptographic information. CBOM generation is still experimental and limited to Java and Python programming languages only, but it allows algorithms, keys, and protocols to be included as `cryptographic-asset` components within a standard CycloneDX BOM. The advantage of *cdxgen* is its integration with the broader CycloneDX ecosystem, which may make it suitable for organizations already using CycloneDX for SBOM workflows.

2.5.3 GitHub CodeQL CBOM Action

GitHub has also introduced an automated way to generate CBOMs using its *CodeQL* analysis engine. CodeQL is a system that can scan source code for patterns, including the use of cryptographic functions. By applying a set of prepared queries, it can detect which algorithms, key sizes, and modes of operation are used in a project [8].

This capability is available as the *cbom-action* [7], a ready-to-use GitHub Action that can be added to the workflow of a software project. When triggered, it scans the repository, runs the cryptography queries, and produces a CBOM in the CycloneDX format. The action supports several programming languages, including Java, Python, JavaScript/TypeScript, Go, C/C++, C#, Swift, and Ruby. The generated CBOM files are attached to the workflow run and can be downloaded directly.

The main strength of this approach is that it works automatically inside the development pipeline, producing an up-to-date CBOM every time the project is built. However, the action is focused on generation only: it does not include a viewer, compliance checks, or storage features. For this reason, it is best understood as a convenient discovery and generation tool that can be combined with other platforms, such as CBOMkit, for deeper analysis and long-term management.

2.5.4 Rationale for Choosing CBOMkit

For this thesis, CBOMkit was selected because it is the only toolchain that provides a complete solution for cryptographic inventories. While other tools exist, they either focus on SBOMs with partial CBOM support or limit themselves to raw discovery of cryptographic usage. CBOMkit, by contrast, integrates every stage of the process: source code scanning, standards-compliant CBOM generation, visualization through a dedicated web interface, compliance checking, and long-term storage.

CycloneDX *cdxgen* can generate CBOMs, but its primary focus remains on SBOMs, and its cryptographic modeling is still basic. GitHub's *cbom-action*, built on CodeQL, is able to scan code and automatically generate CBOMs within continuous integration workflows. However, it is centered on generation alone, without providing a viewer, compliance analysis, or management features. Both tools therefore address important aspects of the problem, but neither offers the complete environment needed for deeper analysis and lifecycle management.

CBOMkit stands out as the only solution capable of analyzing source code directly (not just compiled binaries), offering a viewer for interactive exploration, and a compliance engine to evaluate CBOMs against defined policies, including PQC readiness. Combined with active maintenance by IBM Research, open-source availability, and alignment with NIST's transition strategy, these characteristics made CBOMkit the most suitable and future-proof choice for this thesis.

3 Analysis and compliance

3.1 Introduction

In the original version of CBOMkit, compliance analysis was limited to a single, hardcoded policy embedded within the application logic. This policy focused exclusively on quantum-safe cryptographic practices and required source code modifications to be altered or extended.

This thesis introduces support for externally defined, uploadable policies written in TOML format. Each policy specifies a set of compliance levels and associated rules, which are dynamically loaded and mapped to the cryptographic assets contained in a previously generated CBOM. This decouples policy logic from the application code, allowing organizations to define and manage their own compliance criteria.

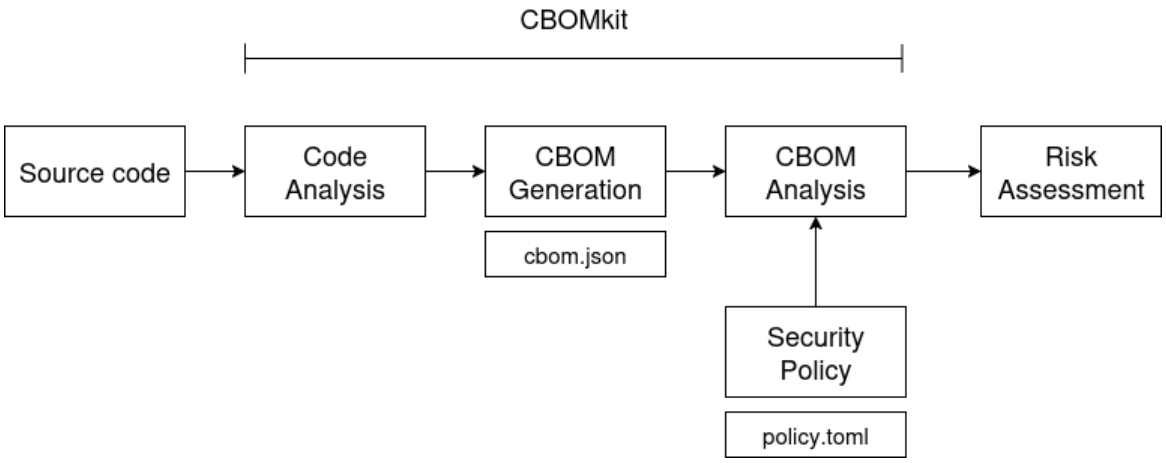

Figure 3.1: Application flow of CBOMkit. Source code is scanned and analyzed to generate a CBOM (`cbom.json`). The CBOM is then evaluated against a security policy (`policy.toml`) to produce compliance results that inform risk assessment.

Cryptographic policies in CBOMkit are now implemented through a structured and extensible configuration system. Each rule describes constraints on cryptographic asset properties, and these are evaluated in real time to classify assets as compliant, non-compliant, or falling under other defined levels.

The source code for this project can be found at `https://github.com/claudioforoncelli/cbomkit` [6].

3.2 Background on Policies and Rules

Cryptographic policies are a set of formalized rules and guidelines established by an organization, institution, or governing body to regulate the use, implementation, and management of cryptographic methods and technologies. These policies define how cryptographic tools, such as encryption, digital signatures, and key management, should be used and applied. Typically, cryptographic policies are structured around compliance levels, which represent predefined levels of cryptographic assurance or strength (e.g., allowed, deprecated, or disallowed), and are used to guide the appropriate use of cryptographic mechanisms across different systems or data types.

A cryptographic policy rule is a specific directive within a broader cryptographic policy that defines

how cryptographic techniques must be applied in a given context. These rules prescribe clear, enforceable requirements for encryption algorithms, key lengths, key management procedures, and usage conditions.

Cryptographic policies are critical for ensuring consistent application of security controls, reducing the risk of human error, and maintaining trust in digital systems. They help organizations meet compliance requirements, protect sensitive data, and respond effectively to emerging threats and technological changes.

Some examples of cryptographic policies are the *NIST Special Publication 800* [20], the *SOG-IS Crypto Evaluation Scheme* [24], or the *ANSSI Guide des Méchanismes Cryptographiques* [1].

3.2.1 TOML as the Configuration Language for CBOMkit Policies

TOML (Tom's Obvious, Minimal Language) is a configuration file format designed to be minimal, easy to read, and semantically unambiguous. As described in its official specification [23], its structure maps naturally to key–value pairs and supports hierarchical data, making it suitable for a broad range of configuration use cases.

The TOML format was selected for representing external policies in CBOMkit to ensure that policy files can be easily authored and maintained even by users without programming expertise. Its clear, human-readable syntax provides the flexibility required to accommodate the structural and semantic needs of the extended software. Alternative formats that were considered during the design process were CSV, which was discarded due to the redundancy it would introduce, as many rows would need to be repeated with minimal variation, and JSON, which was excluded on the basis that its syntax is less intuitive for non-technical users.

3.3 Policy structure

In CBOMkit, the policy file acts as a blueprint for how cryptographic assets, as extracted from a CBOM, are evaluated against a set of predefined compliance rules. The structure of the policy is intentionally designed to be clear, modular, and extensible, allowing non-developers, such as security auditors or compliance officers, to author and maintain these files without requiring changes to the application code.

At the highest level, a CBOMkit policy can be thought of as a hierarchical document with four major domains. These four domains define the policy from the highest level of general metadata down to the lowest level of specific matching conditions.

1. **Policy Header**
   This is the general "header" of the policy, containing global attributes such as the policy's name and id, and the default assessment level to apply in case the analysis of the assets gives an undefined result.

2. **Compliance Levels**
   Compliance levels are security categories that rules assign to matching assets. These levels are asset-specific and policy-dependent, and can be as granular as needed. For example, a cryptographic policy might define levels for "Allowed", "Legacy Use", "Disallowed", "Experimental", and "Unknown". This mapping allows the compliance engine to produce clear, human-readable outputs. Each compliance level is linked to an assessment level, so that the severity of the classification is consistent across the entire evaluation.

3. **Assessment Levels**
   Assessment levels provide a generalized severity scale that captures the overall outcome of a compliance evaluation. They act as an aggregated indicator of the security of the analyzed software, condensing detailed findings into a higher-level view. This abstraction allows results

to be communicated in a clear and comparable way, regardless of the specific policies or rules applied during the evaluation.

4. **Rules**
   The rules section is where the actual matching logic of the policy resides. Each rule defines a set of constraints on the properties of cryptographic assets. When an asset matches all of the constraints defined in a rule, that rule's compliance level is assigned. Rules target specific asset types, for example algorithms, certificates, protocols, or related cryptographic material, and can constrain a wide variety of fields which follow the CBOM structure described in Section 2.4.

3.3.1 Policy Header

The policy metadata acts as the header section of a TOML policy file, providing global information about the policy itself. This data is used by CBOMkit both for identifying the policy and for determining default behavior during evaluation.

Every policy needs a unique `id` so it can be referenced by the program. The `name` field is a human-readable label, used in logs, reports, or dashboards.

The `default_assessment_level` specifies the default assessment level to be used when the global evaluation of all assets does not produce a defined severity. This ensures that even if no compliance levels can be mapped to an assessment level, for example, if no rules match any assets, the policy will still yield a meaningful global security classification.

Example TOML snippet – Policy Header

This example defines a policy header with a unique `id`, display name, and a default assessment level to be used if no matching rules are found.

```
id = "custom_policy"
name = "Custom Cryptographic Compliance Policy"
default_assessment_level = 1
```

3.3.2 Assessment Levels

Assessment levels represent the global severity scale used by the policy to summarize the overall security and compliance posture of the software under analysis. They are not tied to a single asset, but rather provide an aggregated view across all cryptographic assets examined in a CBOM.

The purpose of assessment levels is to produce a concise, human-readable classification of the analyzed software. This is useful for compliance reporting, dashboards, and automated decision-making processes that depend on a clear indication of security status.

A key distinction between assessment levels and compliance levels is that assessment levels operate at the policy-wide level, while compliance levels are applied per asset. This mapping allows the compliance engine to translate multiple, potentially heterogeneous asset-specific classifications into a single global result.

For example, a policy may define three assessment levels:

- 1 – Compliant: All assets conform to the defined rules.
- 2 – Potentially compliant: Some assets are compliant while others are deprecated or unknown.
- 3 – Non-compliant: One or more assets use algorithms or configurations explicitly disallowed by the policy.

When the evaluation is complete, the engine determines the worst assessment level reached by any asset's compliance level mapping. This is done by iterating over all evaluated assets, looking up the

assessment level associated with each asset's compliance level, and comparing it to the current worst value. The comparison is numeric, where a higher `id` typically corresponds to a more severe state. If the engine finds an assessment level with a higher `id` than the current worst, it replaces the current value.

This iterative process ensures that the final assessment level reflects the most critical non-compliance detected among all assets. The resulting value is then assigned as the final assessment level for the entire policy.

In CBOMkit's TOML policy format, assessment levels are defined as a top-level array of objects. Each object must include:

- `id`: a unique integer identifier.
- `label`: a short, human-readable description (e.g., "Compliant").

Example TOML snippet – Assessment Levels definition

This example defines three assessment levels, from fully compliant to non-compliant.

```
[[assessment_levels]]
id = 1
label = "Compliant"

[[assessment_levels]]
id = 2
label = "Potentially compliant"

[[assessment_levels]]
id = 3
label = "Not compliant"
```

3.3.3 Compliance Levels

Compliance levels represent the classification assigned to each individual cryptographic asset after it has been evaluated against the policy rules. Unlike assessment levels, which provide a single global indicator for the entire policy evaluation, compliance levels operate at the asset-specific level.

Each compliance level in a policy is linked to exactly one assessment level via its `assessment_level` field. This link enables the compliance engine to translate fine-grained, asset-specific categorizations into a unified global severity result.

The set of compliance levels defined in a policy is entirely customizable, allowing organizations to express the granularity of their security classifications.

Each compliance level definition includes:

- `id`: a unique integer identifier for the level.
- `label`: a short, human-readable name for the level.
- `description`: an optional text explaining the rationale or scope of the level.
- `color`: a visual indicator for dashboards and reports.
- `icon`: a symbolic representation for UI purposes.
- `assessment_level`: the identifier of the assessment level to which this compliance level maps.

Example TOML snippet – Compliance Levels definition

This example defines three compliance levels, each linked to one of the assessment levels defined above.

```
[[compliance_levels]]
id = 1
label = "Acceptable"
description = "Approved for use"
color = "green"
icon = "CHECKMARK"
assessment_level = 1

[[compliance_levels]]
id = 2
label = "Deprecated"
description = "Use is discouraged and may be disallowed soon"
color = "#ffc107"
icon = "WARNING"
assessment_level = 2

[[compliance_levels]]
id = 3
label = "Disallowed"
description = "Use is disallowed"
color = "#dc3545"
icon = "ERROR"
assessment_level = 3
```

3.3.4 Rules

The rules section defines the actual matching logic of the policy. Each rule describes one or more constraints that an asset must meet to be assigned a specific compliance level.

Every rule must include the `asset_type` field, which is both mandatory and critical to the rule's definition. The `asset_type` determines which set of attributes are valid and available for matching. This is because different types of cryptographic assets expose different properties in the CBOM structure. For example:

- An `ALGORITHM` asset can specify fields such as `primitive`, `mode` (mode of operation), `padding`, or `cryptoFunctions`.
- A `CERTIFICATE` asset, instead, exposes fields like `issuerName`, `subjectName`, `notValidAfter`, and `signatureAlgorithmRef`, but it does not have attributes such as `mode` or `primitive`.
- A `PROTOCOL` asset includes `type`, `version`, and potentially a list of `cipherSuites`.

Because of this, specifying the correct `asset_type` in the rule is essential: it not only filters which assets the rule will be applied to, but it also dictates the set of fields that can be meaningfully used in the rule's matching conditions.

The fields available in a rule directly mirror the attributes of the corresponding `CryptoProperties` subtype. This ensures that policy authors can specify constraints using the same terminology and structure as the CBOM asset definitions described in Section 2.4.

Each rule definition contains:

- `name`: An optional string used to match the name of the cryptographic asset.

- `compliance_level`: The ID of the compliance level to assign if the rule matches.
- `description`: A human-readable explanation of the rule's purpose.
- `asset_type`: The type of asset this rule applies to (`ALGORITHM`, `CERTIFICATE`, `PROTOCOL`, `RELATED_CRYPTO_MATERIAL`).
- One or more matching fields from the asset's property structure.

3.3.5 Rule Matching and Precedence

During evaluation, an asset matches a rule if **all** non-null fields specified in the rule exactly match the corresponding properties of the asset. In cases where multiple rules apply to the same asset, CBOMkit resolves conflicts using a precedence mechanism.

This mechanism selects the rule with the highest specificity, meaning the rule that defines the greatest number of matching constraints on the asset's properties. If two rules have equal specificity, the one with the higher compliance level identifier is chosen.

Example TOML snippet – Rules for algorithms

This example defines three rules targeting cryptographic algorithms: one deprecating SHA-1, one approving AES-128, and one disallowing ECB mode for encryption.

```
[[rule]]
name = "sha1"
compliance_level = 4
description = "SHA-1 is deprecated and disallowed after 2030"
asset_type = "ALGORITHM"

[[rule]]
name = "AES128"
compliance_level = 2
description = "AES128 is acceptable"
asset_type = "ALGORITHM"

[[rule]]
mode = "ecb"
compliance_level = 5
crypto_functions = ["ENCRYPT"]
description = "ECB encryption is disallowed"
asset_type = "ALGORITHM"
```

3.4 Implementation of the Policy Evaluation Engine

Section 3.3 describes the structure and semantics of CBOMkit policies from a configuration perspective. We now turn to the internal Java implementation that enables these policies to be loaded, validated, and applied to cryptographic assets at runtime.

The architecture follows a modular pipeline (Figure 3.2). A policy file written in TOML is parsed into a `CustomCompliancePolicy`, which contains metadata, assessment levels, compliance levels, and rules. This object is then wrapped into a `CustomComplianceService`, a generic evaluation engine that applies the rules to CBOM assets. Finally, the service is registered with the `ComplianceServiceSelector`, which manages multiple available policies and ensures that the correct one is used during evaluation.

The following sections describe these components in detail, covering the service selector, the policy parser, and the evaluation service.

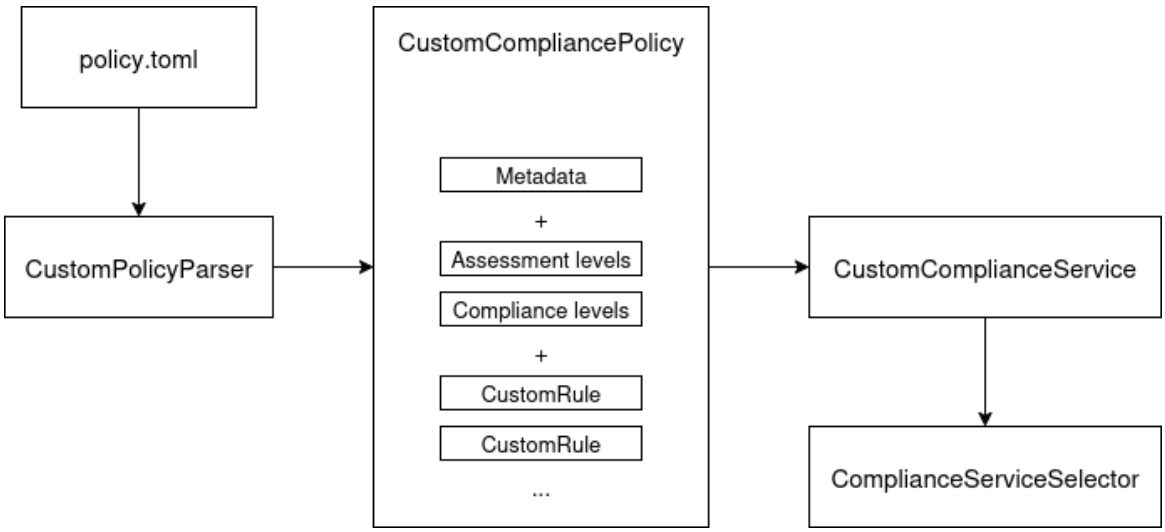

Figure 3.2: Java implementation flow of CBOMkit's compliance engine.

3.4.1 The Compliance Service Selector

The `ComplianceServiceSelector` class acts as the policy router for the entire CBOMkit evaluation system. It maintains a registry of available policy instances and exposes methods to retrieve and register them dynamically. This enables the system to support both statically defined policies, compiled into the codebase, and externally defined ones uploaded by the user in TOML format.

Policies are registered using string identifiers (e.g., `"quantum_safe"`), which are mapped to their corresponding service implementation. When a compliance check is initiated, the selector resolves the requested policy identifier and returns the appropriate evaluation engine. Dynamically registered policies are stored in memory for the current execution of the application, ensuring that they are immediately available for evaluation without requiring a restart.

Class Overview

A simplified excerpt of the `ComplianceServiceSelector` is shown below. The complete implementation is provided in Appendix A.1, Listing A.1.

```
@ApplicationScoped
public class ComplianceServiceSelector {
    private final Map<String, IComplianceService> services = new HashMap<>();
    private IComplianceService defaultService;

    @PostConstruct
    public void init() {
        services.put("quantum_safe", new BasicQuantumSafeComplianceService());
        this.defaultService = services.get("quantum_safe");
    }
}
```

The use of a central selector class offers multiple advantages. It enhances extensibility by allowing new policies to be loaded at runtime without requiring modifications to the application code. It promotes separation of concerns, as the selection of the policy implementation is decoupled from the evaluation logic, ensuring a cleaner and more maintainable architecture. Finally, it improves configurability, enabling administrators and users to upload TOML-based policies through a web interface or API, with the system dynamically registering them for immediate use.

Once registered, these dynamic policies are treated identically to built-in policies during evaluation, allowing seamless integration of custom rule sets into the compliance workflow.

3.4.2 The Custom Policy Parser

User-defined TOML policy files must be transformed into in-memory Java objects before they can be evaluated against CBOM assets. This task is handled by the `CustomPolicyParser`, which reads the TOML configuration, validates its structure, and instantiates a corresponding `CustomCompliancePolicy` object containing one or more `CustomRule` instances.

The parser ensures that the loaded policy conforms to the structure described in Section 3.3, checking that all mandatory top-level sections (`id`, `name`, `default_assessment_level`, `assessment_levels`, `compliance_levels`, and `rules`) are present. This validation step prevents the evaluation engine from encountering incomplete or malformed policies at runtime.

CustomCompliancePolicy

The `CustomCompliancePolicy` class is the in-memory representation of the entire TOML policy. It stores:

- Metadata from the policy header (`id`, `name`, `default_assessment_level`).
- A list of assessment level objects, each containing an `id` and `label`.
- A list of compliance level objects, each containing classification details and a mapping to an assessment level.
- A list of rules (`CustomRule` instances) defining asset-specific constraints.

This object acts as the primary data source for the evaluation engine, which queries its lists and mappings when analyzing each cryptographic asset.

CustomRule

Each `CustomRule` object represents a single rule from the TOML file. Its fields mirror the CBOM asset properties relevant to the specified `asset_type`. This one-to-one mapping between rule fields and CBOM attributes ensures that policy authors can use familiar terminology and guarantees compatibility with the evaluation logic.

A `CustomRule` contains:

- `name` (optional): name match for the cryptographic asset.
- `compliance_level`: ID of the compliance level to assign if the rule matches.
- `description`: a human-readable explanation of the rule.
- `asset_type`: the type of asset the rule applies to.
- One or more attribute constraints (e.g., `primitive`, `mode`, `crypto_functions` for algorithms; `issuerName`, `notValidAfter` for certificates).

Parsing Flow

The parsing process follows these steps:

1. Load the TOML file from disk or receive it from an API upload.
2. Use a TOML parsing library to convert the file into a nested data structure.
3. Map the top-level fields into a new `CustomCompliancePolicy` instance.
4. Map each `[assessment_levels]` and `[compliance_levels]` entry into their respective objects and add them to the policy.

5. Map each `[rule]` entry into a `CustomRule` object, ensuring that the `asset_type` is valid and that only the appropriate attributes for that type are set.
6. Perform validation checks (e.g., all referenced compliance levels exist; assessment level IDs are unique).

If the policy passes validation, the resulting `CustomCompliancePolicy` object is registered with the `ComplianceServiceSelector` by wrapping it in a `CustomComplianceService` instance.

Example Code Snippet

Below is an excerpt of the `CustomPolicyParser`, which loads and validates TOML policies before instantiating in-memory `CustomCompliancePolicy` objects. The full implementation is provided in Appendix A.2, Listing A.2.

```
public class CustomPolicyParser {

    public static CustomCompliancePolicy parse(String tomlString) {
        TomlParseResult toml = Toml.parse(tomlString);

        // ---- Policy Header ----
        CustomCompliancePolicy policy = new CustomCompliancePolicy();
        policy.setId(requireString(toml, "id"));
        policy.setName(requireString(toml, "name"));
        policy.setDefaultLevel(Math.toIntExact(
            requireLong(toml, "default_assessment_level")));

        // ... shortened: assessment levels, compliance levels, rules ...

        return policy;
    }
}
```

3.4.3 The Custom Compliance Service

Once a policy has been successfully parsed into a `CustomCompliancePolicy` object, it must be evaluated against the cryptographic assets extracted from the CBOM. This task is performed by the `CustomComplianceService` class, which acts as a generic rule evaluation engine for user-defined policies. The service is responsible for providing access to the policy's metadata, compliance levels, and default values, iterating through all cryptographic assets and matching them against the policy rules, determining the compliance level of each asset and mapping it to the corresponding assessment level, and finally aggregating the results to identify the worst assessment level across all analyzed assets.

Evaluation Flow

The evaluation process follows a deterministic sequence:

1. Initialize the worst assessment level to the policy's default assessment level.
2. For each asset in the CBOM:
   (a) Iterate over all rules in the policy that target the asset's type.
   (b) Check if the asset matches the rule's constraints on its properties.
   (c) If matched, determine the rule's *specificity* (i.e., how many properties it defines) and select the most specific matching rule.
   (d) Assign the compliance level of the selected rule to the asset.
   (e) Map the compliance level to its associated assessment level.

(f) Update the worst assessment level if the mapped level is more severe than the current one.

3. Once all assets are processed, return a result object containing:
   - A list of per-asset compliance results.
   - The final worst assessment level.

This design ensures that the evaluation outcome is both granular (per asset) and aggregated (policy-wide), enabling use cases such as detailed reporting and high-level dashboard summaries.

Rule Matching and Specificity

A core feature of the `CustomComplianceService` is its ability to resolve conflicts when multiple rules match the same asset. In such cases, the engine chooses the rule with the highest specificity, calculated as the number of non-null matching fields in the rule. If two rules have the same specificity, the one with the higher compliance level ID is preferred. This mechanism guarantees deterministic and transparent rule selection.

Example Code Snippet

The code below shows an excerpt of the `CustomComplianceService`, focusing on its evaluation loop and rule matching process. The full implementation is provided in Appendix A.1, Listing A.3.

```
public class CustomComplianceService implements IComplianceService {

    private final CustomCompliancePolicy policy;

    public ComplianceCheckResultDTO evaluate(
        PolicyIdentifier policyIdentifier,
        Collection<CryptographicAsset> assets) {

        List<ICryptographicAssetPolicyResult> results = new ArrayList<>();
        AssessmentLevel worstAssessment = getDefaultAssessmentLevel();

        for (CryptographicAsset asset : assets) {
            ICryptographicAssetPolicyResult result = evaluate(asset);
            results.add(result);
            // ... compliance/assessment mapping logic ...
        }

        return new ComplianceCheckResultDTO(results, false, worstAssessment);
    }

    private ICryptographicAssetPolicyResult evaluate(CryptographicAsset asset) {
        int highestSpecificity = -1;
        ComplianceLevel bestLevel = getDefaultComplianceLevel();
        // ... rule matching logic ...
        return new BasicCryptographicAssetPolicyResult(
            asset.identifier().toLowerCase(), bestLevel, "Matched␣rule");
    }
}
```

The `CustomComplianceService` is intentionally generic: it can evaluate any policy conforming to the TOML structure previously described, without requiring custom logic for each policy. This design decouples the policy definition from the evaluation logic, allowing policies to be authored, updated, and deployed without recompiling the software.

4 Results

4.1 Introduction

This chapter presents the results obtained by applying the extended CBOMkit compliance engine to real-world cryptographic inventories. In particular, two CBOMs were analyzed: a synthetic test CBOM designed to validate the correctness of the evaluation logic, and the CBOM generated for Keycloak [16, 15], a widely used open-source identity and access management solution. Both analyses were performed using a custom policy authored for this work, which was inspired by the recommendations of a NIST guideline.

4.2 Policy Basis

The custom policy used in these experiments is directly inspired by the compliance requirements defined in NIST SP 800-131Ar3 [20]. This standard establishes a framework for classifying the acceptability of cryptographic assets, defining compliance levels that indicate whether an algorithm, key length, or parameter set is suitable for use in secure systems.

The compliance levels defined by NIST SP 800-131Ar3 are:

- Acceptable: algorithms and key lengths/strengths that are fully approved for use according to NIST recommendations and related guidance.
- Deprecated: algorithms or parameters that may still be used, but with caution or only under certain conditions. Their use introduces security risk and requires risk owners to assess the implications carefully.
- Disallowed: algorithms, key sizes, parameter sets, or schemes that are no longer permitted for use.
- Legacy use: algorithms or parameters that cannot be used for protecting new data, but may still be applied to process information that was already protected in the past. For instance, decrypting existing ciphertext or verifying digital signatures generated before the algorithm was disallowed.

In addition to defining these categories, the policy also specifies detailed rules that govern multiple types of cryptographic assets. These rules translate the compliance levels into actionable checks, ensuring they can be applied consistently across a broad range of cryptographic material. The categories include:

- Algorithms: symmetric ciphers (AES, 3DES), hash functions (SHA-1, SHA-256), and asymmetric schemes (RSA, ECDSA).
- Encryption modes: for example, ECB (disallowed) or GCM (acceptable).
- Key properties: including minimum security thresholds, such as RSA $\geq$ 2048 bits.
- Certificates: checking signature algorithms, such as disallowing certificates signed with SHA-1.
- Protocols: validating that only secure versions and cipher suites are permitted, for example TLS 1.2+ with approved ciphers.

While NIST defines compliance levels and their application to individual assets, it does not specify how to aggregate these results into a unified evaluation of an entire software system. To address this limitation, assessment levels were introduced to summarize the overall compliance posture of a complete CBOM. These assessment levels map multiple asset-specific classifications into a single global result.

The assessment levels chosen for this policy are defined as follows:

- Compliant: all assets in the CBOM are classified as acceptable under the policy.
- Potentially Non-Compliant: at least one asset is classified as legacy or deprecated, while no assets are disallowed.
- Non-Compliant: one or more assets are classified as disallowed.

4.3 Evaluation of the Test CBOM

To make the evaluation concrete, this subsection focuses on three assets extracted from the test CBOM: `AES128-ECB-PKCS5` (ECB encryption), `SHA1`, and `3DES`.

CBOM Excerpt

An excerpt from the test CBOM is shown below. The complete JSON file is available in Appendix B.2, Listing B.2.

```
{
  "type": "cryptographic-asset",
  "name": "AES128-ECB-PKCS5",
  "cryptoProperties": {
    "assetType": "algorithm",
    "algorithmProperties": {
      "primitive": "block-cipher",
      "mode": "ecb",
      "cryptoFunctions": ["decrypt","encrypt"]
    }
  }
}
```

Relevant Policy Rules (Excerpt)

An example rule is shown below. The full TOML ruleset is available in Appendix B.1, Listing B.1.

```
[[rule]]
name = "sha1"
compliance_level = 4
description = "SHA-1 is deprecated and disallowed after 2030"
asset_type = "ALGORITHM"
```

Matching and findings

AES128-ECB-PKCS5 The asset is an algorithm with `mode="ecb"` and `cryptoFunctions=["decrypt","encrypt"]`. Within the policy, several rules govern ECB usage: one applies to ECB mode in general (without specifying functions), while others refine the condition to encryption-only, decryption-only, or both. Technically, both the generic ECB rule and the function-specific rules match this asset. However, CBOMkit resolves such overlaps by preferring the *most specific* rule, defined as the one that constrains the largest number of attributes. In this case, the rule requiring both `ENCRYPT` and `DECRYPT` explicitly covers all of the asset's functions, making it strictly more specific than the generic rule or the single-function rules. As a result, this rule is selected, and the

asset is classified with compliance level 5 (*disallowed*). The viewer reports this classification as "Use is disallowed" with the rationale "ECB mode is disallowed for encryption (Section 2.2)." This example illustrates how CBOMkit handles overlapping matches by always favoring the rule with the greater number of matching parameters, thereby ensuring that the evaluation reflects the precise context of the asset.

SHA1 The asset is an algorithm named `SHA1`. It matches the rule by `name="sha1"`, yielding compliance level 4 (deprecated). The viewer shows: "Use is discouraged and may be disallowed soon," with rationale tied to the scheduled disallowance after 2030 (Section 1.2).

3DES The asset is an algorithm named `3DES` with `primitive="block-cipher"`. It matches the rule by `name="3des"`, yielding compliance level 5 (disallowed). The viewer reports: "Use is disallowed" with rationale "TDEA (3DES) encryption is disallowed (Section 2.1)."

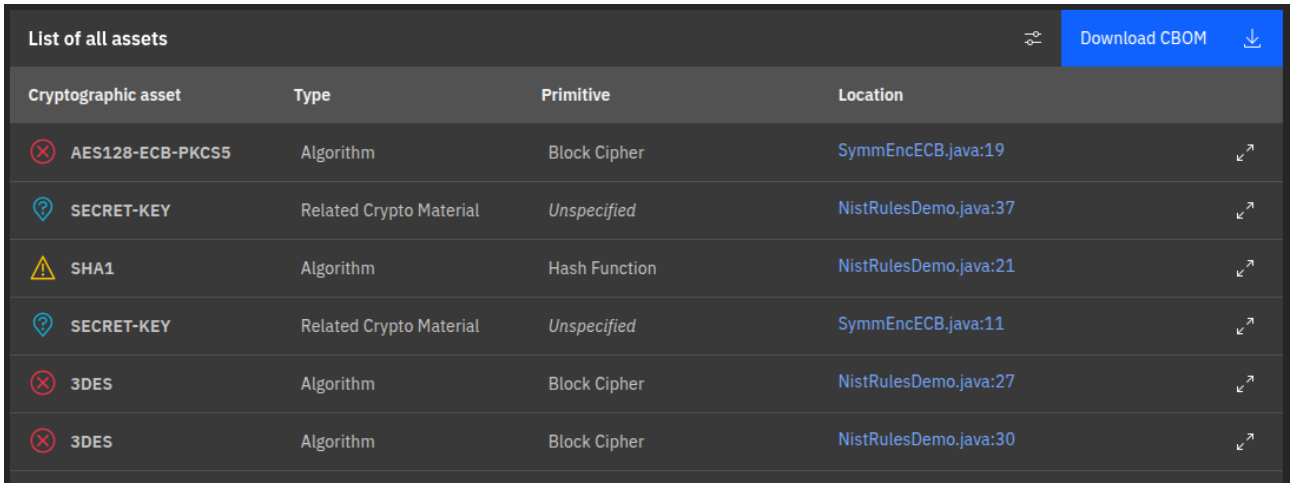

Figure 4.1: Viewer output for the test CBOM. Assets such as AES128-ECB-PKCS5, SHA-1, and 3DES are flagged according to the rules defined in the compliance policy.

Global assessment

Under the CBOMkit assessment scheme, the presence of any disallowed asset makes the overall result *Non-Compliant*. Since the test CBOM contains both ECB encryption and 3DES (disallowed), alongside SHA-1 (deprecated), the final assessment for this CBOM is *Non-Compliant*.

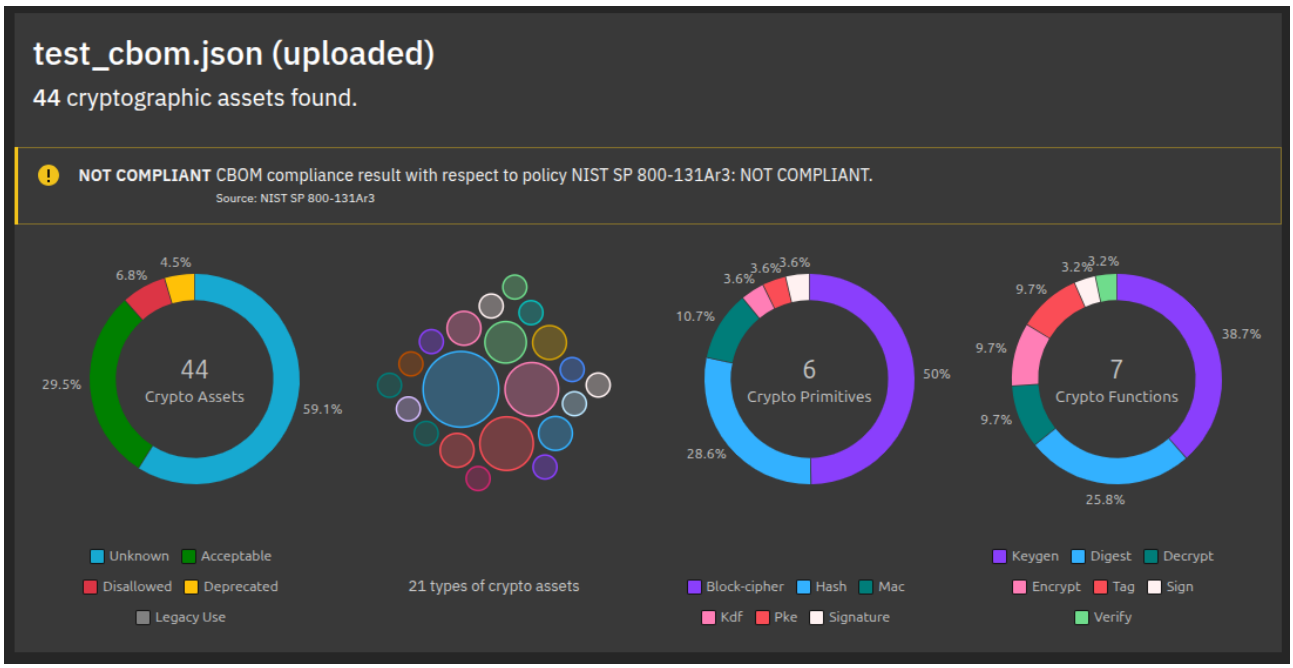

Figure 4.2: CBOMkit viewer summary for the test CBOM. The dashboard shows the global assessment result (*Non-Compliant*) together with statistics on cryptographic assets, primitives, and functions detected.

4.4 Evaluation of the Keycloak CBOM

After validating the engine with the test CBOM, the same policy was applied to the CBOM generated from the software Keycloak [16], an open-source identity and access management system widely used in enterprise contexts. The version analyzed corresponds to the `main` branch at commit `c5b391d` (26 February 2025) [17]. Despite its maturity and professional maintenance, CBOMkit successfully revealed the presence of deprecated cryptographic primitives within the codebase.

Highlighted Asset: SHA-1

An excerpt from the Keycloak CBOM showing a SHA-1 occurrence is shown below. The complete JSON fragment is available in Appendix B.3, Listing B.3.

```
{
  "type": "cryptographic-asset",
  "name": "SHA1",
  "evidence": {
    "occurrences": [
      {
        "location": "server-spi-private/.../ArtifactBindingUtils.java",
        "line": 45
      }
    ]
  }
}
```

The corresponding policy rule is:

```
[[rule]]
name = "sha1"
compliance_level = 4
description = "SHA-1 is deprecated and disallowed after 2030 (Section 1.2)"
asset_type = "ALGORITHM"
```

This match resulted in the asset being classified as *Deprecated*. In the viewer, this appears as: "Use is discouraged and may be disallowed soon," with the justification that SHA-1 will be disallowed after 2030 according to NIST SP 800-131Ar3. The resulting global assessment level for the Keycloak CBOM was therefore *Potentially Non-Compliant*.

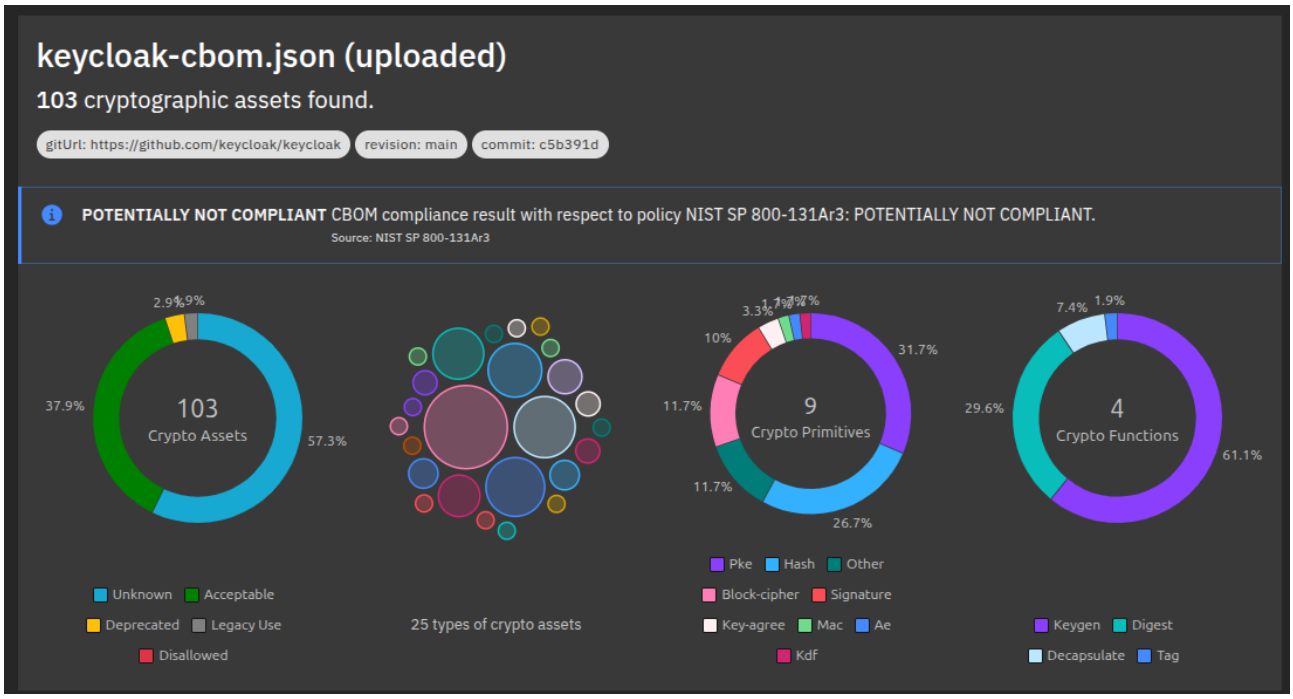

Figure 4.3: CBOMkit viewer analysis of the Keycloak CBOM. Out of 103 detected cryptographic assets, the overall result is *Potentially Not Compliant* due to the presence of deprecated primitives.

Algorithm

SHA1

Code

View code

```
41   * @return SHA-1 hash of the given identifierFrom
42   */
43  public static byte[] computeArtifactBindingIdentifier(String identifierFrom) {
44      try {
45          MessageDigest sha1Digester = MessageDigest.getInstance("SHA-1");
46          return sha1Digester.digest(identifierFrom.getBytes(StandardCharsets.UTF_8));
47      } catch (NoSuchAlgorithmException e) {
48          throw new RuntimeException("JVM does not support required cryptography algorithms: SHA-1/SHA1PRNG.", e);
49      }
```

Compliance

Use is discouraged and may be disallowed soon

Policy: NIST SP 800-131Ar3

Compliance Information	Category
- SHA-1 is deprecated and disallowed after 2030 (Section 1.2)	Deprecated

Specification

Type	Value
Asset Type	Algorithm
Primitive	Hash Function
Parameter Set Identifier	160
Crypto Functions	Digest
OID	1.3.14.3.2.26
BOM Reference	c6a76fd5-c091-4553-a157-79850036cfec

Figure 4.4: Detailed viewer report of a SHA-1 instance found in Keycloak. The snippet highlights the exact code location, with compliance status set to *Deprecated* under NIST SP 800-131Ar3.

The detection of SHA-1 in Keycloak highlights the value of automated CBOM analysis. Even professional, widely deployed systems may contain cryptographic components that are no longer recommended, and which could compromise long-term security and compliance.

4.5 Discussion

The results highlight both the accuracy and the practical value of the extended CBOMkit framework. From a methodological perspective, the test CBOM confirmed that the compliance engine correctly handles rule overlaps, always selecting the most specific match to avoid ambiguous classifications. This feature is important in real-world contexts where assets may satisfy multiple rules simultaneously.

Applying the policy to Keycloak demonstrated the framework's utility beyond controlled tests. Despite Keycloak being a professionally maintained project, the analysis surfaced the continued presence of SHA-1, a deprecated primitive with known long-term risks. Such findings illustrate that automated CBOM-based compliance checking can expose residual weaknesses that might otherwise remain unnoticed in manual audits.

5 Conclusion and Future Work

5.1 Summary of Contributions

This thesis addressed the problem of evaluating the compliance of cryptographic components in software systems. As organizations face growing regulatory demands and prepare for the post-quantum transition, it is important to maintain precise visibility and enforceable rules for cryptographic usage.

The work began with an investigation of Cryptography Bills of Materials, covering their role in risk management, the standards defining their structure, and the tools available for generating and analyzing them. CBOMkit was chosen as the basis due to its open-source availability, dedicated focus on cryptographic inventories, and integration of generation, visualization, and compliance features.

CBOMkit was extended with a structured, machine-readable policy format in TOML, capturing metadata, compliance levels, assessment levels, and rules. On top of this, a Java evaluation framework was built, including a policy parser and an evaluation engine, which together allow both built-in and user-defined policies to be applied in a flexible way.

Validation on a test CBOM and the CBOM of Keycloak, a real-world identity and access management platform, showed that the system correctly flags deprecated or disallowed algorithms (e.g., SHA-1, 3DES, AES/ECB) and propagates these findings into a system-level compliance posture through assessment levels.

5.2 Future Work

While this thesis establishes a solid foundation for cryptographic compliance evaluation, several opportunities remain. Current analysis depends on CBOMs from static analyzers with limited language and library coverage (mainly Java and Python). Many cryptographic usages, especially in custom routines or niche libraries, may remain undetected by static approaches.

A promising path is the use of large language models (LLMs) fine-tuned on code. Such models could detect cryptographic function calls across diverse languages (C++, Go, Rust, JavaScript), extract metadata such as algorithm, mode, and key length, and build CBOMs directly from source code. This would extend CBOMkit's reach, enabling end-to-end inventory and compliance evaluation for heterogeneous codebases.

Future research should survey related techniques, design LLM-based extraction strategies, prototype an end-to-end pipeline, and benchmark accuracy against static analyzers. Together, this would complement the present work: CBOMkit provides the compliance framework, while LLM-driven analysis could strengthen CBOM generation, yielding a more comprehensive solution.

Closing Remarks

This thesis demonstrates that cryptographic compliance can be expressed and enforced through a structured, machine-readable framework, turning high-level standards into systematic evaluations. More importantly, it highlights the role of cryptographic inventories as the basis for compliance, risk management, and long-term security. In the context of the post-quantum transition, this principle becomes especially clear: you cannot migrate what you do not know.

Bibliography

[1] Agence nationale de la sécurité des systèmes d'information (ANSSI). Mécanismes cryptographiques: Règles et recommandations. Technical report, ANSSI, January 2020.

[2] Common Criteria Recognition Arrangement (CCRA). Common criteria for information technology security evaluation (ISO/IEC 15408). Technical report, ISO/IEC and CCRA, April 2022.

[3] CycloneDX Project. `cdxgen`: Cyclonedx generator, github repository. `https://github.com/CycloneDX/cdxgen`.

[4] CycloneDX Project. `cdxgen` advanced functionality: Generate cryptography bill of materials (CBOM). `https://cyclonedx.github.io/cdxgen/#/ADVANCED?id=generate-cryptography-bill-of-materials-cbom`, 2025.

[5] European Union Agency for Cybersecurity (ENISA). Post-quantum cryptography: Current state and quantum mitigation. Technical report, ENISA, May 2021.

[6] Claudio Foroncelli. CBOMkit: Extended cryptography bill of materials toolkit. `https://github.com/claudioforoncelli/cbomkit`.

[7] GitHub Advanced Security. CBOM action: Generate cryptography bill of materials with CodeQL. `https://github.com/advanced-security/cbom-action`.

[8] GitHub Security Lab. Addressing post-quantum cryptography with CodeQL. `https://github.blog/security/vulnerability-research/addressing-post-quantum-cryptography-with-codeql/`, December 2023.

[9] IBM Research. CBOMkit: An open-source toolkit for cryptography bills of materials. `https://github.com/IBM/cbomkit`.

[10] IBM Research. Sonar cryptography: Detection rule structure. `https://github.com/PQCA/sonar-cryptography/blob/main/docs/DETECTION_RULE_STRUCTURE.md`.

[11] IBM Research. Sonar cryptography: Supported languages. `https://github.com/PQCA/sonar-cryptography/blob/main/docs/LANGUAGE_SUPPORT.md`.

[12] IBM Research. Quantum-safe cryptography with CBOMkit, November 2024. IBM Research Blog.

[13] ISO/IEC JTC 1. ISO/IEC 19790:2012 – information technology – security techniques – security requirements for cryptographic modules. Technical report, International Organization for Standardization (ISO), 2012.

[14] ISO/IEC JTC 1. ISO/IEC 5962:2021 – information technology – software package data exchange (SPDX). Technical report, International Organization for Standardization (ISO), January 2021.

[15] Keycloak Project. Keycloak GitHub repository. `https://github.com/keycloak/keycloak`.

[16] Keycloak Project. Keycloak: Open source identity and access management. `https://www.keycloak.org/`.

[17] Keycloak Project. Keycloak source code analyzed during testing. `https://github.com/keycloak/keycloak/commit/c5b391d`. Commit `c5b391d` on the `main` branch.

[18] National Institute of Standards and Technology (NIST). FIPS PUB 140-3: Security requirements for cryptographic modules. Technical report, U.S. Department of Commerce, March 2019.

[19] National Institute of Standards and Technology (NIST). NIST interagency report (NIST IR) 8547: Cryptographic asset management. Technical Report NIST IR 8547 (Initial Public Draft), National Institute of Standards and Technology (NIST), November 2024.

[20] National Institute of Standards and Technology (NIST). Transitioning the use of cryptographic algorithms and key lengths: NIST special publication 800-131a revision 3 (initial public draft). Technical report, NIST, October 2024.

[21] William Newhouse, Murugiah Souppaya, William Barker, and Christopher Brown. Migration to post-quantum cryptography: Quantum readiness – cryptographic discovery (preliminary draft). Special Publication 1800-38B, National Institute of Standards and Technology (NCCoE), Gaithersburg, MD, December 2023. Preliminary Draft; Released for public comment December 19, 2023.

[22] OWASP Foundation. Authoritative guide to cryptography bill of materials (CBOM). Technical report, OWASP Foundation, April 2024. First Edition.

[23] Tom Preston-Werner. TOML: Tom's obvious, minimal language. `https://toml.io/en/`, 2025. Accessed: 2025-08-29.

[24] Senior Officials Group – Information Systems Security (SOG-IS). SOG-IS crypto evaluation scheme: Supporting documents. Technical report, SOG-IS, February 2023.

[25] Shalanda D. Young. Migrating to post-quantum cryptography. Memorandum M-23-02, Executive Office of the President, Office of Management and Budget (OMB), Washington, DC, November 2022.

Appendix A

Code Listings

This appendix contains the full Java implementation excerpts referenced in Chapter 3. Only shortened snippets were shown in the main text to preserve readability; the complete versions are included here for completeness.

A.1 Compliance Service Selector

Listing A.1: Core logic of `ComplianceServiceSelector`

```
public class ComplianceServiceSelector {
  private final Map<String, IComplianceService> services = new HashMap<>();
  private IComplianceService defaultService;

  @PostConstruct
  public void init() {
    // Instantiate available compliance services
    IComplianceService quantumSafeService = new BasicQuantumSafeComplianceService();

    // Register them with identifiers
    services.put("quantum_safe", quantumSafeService);

    // Define the default service
    this.defaultService = quantumSafeService;
  }

  public IComplianceService getService(String policyIdentifier) {
    IComplianceService service = services.get(policyIdentifier);
    if (service == null) {
      service = defaultService;
    }
    return service;
  }

  public void register(String policyIdentifier, IComplianceService service) {
    services.put(policyIdentifier, service);
  }

  public boolean remove(String policyIdentifier) {
    // Prevent deleting built-in policies
    if (policyIdentifier.equals("quantum_safe")) {
      return false;
    }
    return services.remove(policyIdentifier) != null;
  }

  public List<Map<String, String>> listPolicies() {
    List<Map<String, String>> policyList = new ArrayList<>();
    for (Map.Entry<String, IComplianceService> entry : services.entrySet()) {
```

```
      Map<String, String> map = new HashMap<>();
      map.put("id", entry.getKey());
      map.put("label", entry.getValue().getName());
      policyList.add(map);
    }
    return policyList;
  }
}
```

A.2 Custom Policy Parser

Listing A.2: Shortened implementation of `CustomPolicyParser`

```
public class CustomPolicyParser {
  public static CustomCompliancePolicy parse(String tomlString) {
    TomlParseResult toml = Toml.parse(tomlString);

    CustomCompliancePolicy policy = new CustomCompliancePolicy();
    policy.setId(requireString(toml, "id"));
    policy.setName(requireString(toml, "name"));
    policy.setDefaultLevel(Math.toIntExact(requireLong(toml, "default_assessment_level")));

    List<AssessmentLevel> assessmentLevels = new ArrayList<>();
    TomlArray assessmentLevelsArray = requireArray(toml, "assessment_levels");
    for (int i = 0; i < assessmentLevelsArray.size(); i++) {
      Object entry = assessmentLevelsArray.get(i);
      if (!(entry instanceof TomlTable level)) continue;
      int id = requireInt(level, "id");
      String label = requireString(level, "label");
      assessmentLevels.add(new AssessmentLevel(id, label));
    }
    policy.setAssessmentLevels(assessmentLevels);

    List<ComplianceLevel> levels = new ArrayList<>();
    TomlArray levelsArray = requireArray(toml, "compliance_levels");
    for (int i = 0; i < levelsArray.size(); i++) {
      TomlTable level = levelsArray.getTable(i);
      ComplianceLevel complianceLevel = new ComplianceLevel(
        requireInt(level, "id"),
        requireString(level, "label"),
        requireString(level, "description"),
        requireString(level, "color"),
        parseEnum(level, "icon", ComplianceIcon.class),
        requireInt(level, "assessment_level")
      );
      levels.add(complianceLevel);
    }
    policy.setLevels(levels);

    List<RuleDefinition> rules = new ArrayList<>();
    TomlArray rulesArray = requireArray(toml, "rule");
    for (int i = 0; i < rulesArray.size(); i++) {
      TomlTable table = rulesArray.getTable(i);
      RuleDefinition rule = new RuleDefinition();
      rule.setName(table.getString("name"));
      rule.setDescription(requireString(table, "description"));
      rule.setLevelId(requireInt(table, "compliance_level"));

      CryptoProperties props = new CryptoProperties();
      props.setOid(table.getString("oid"));
      props.setAssetType(parseEnum(table, "asset_type", AssetType.class));

      switch (props.getAssetType()) {
        case ALGORITHM -> {
```

```
    AlgorithmProperties alg = new AlgorithmProperties();
    if (table.contains("primitive")) {
      alg.setPrimitive(parseEnum(table, "primitive", Primitive.class));
    }
    if (table.contains("mode")) {
      alg.setMode(parseEnum(table, "mode", Mode.class));
    }
    if (table.contains("padding")) {
      alg.setPadding(parseEnum(table, "padding", Padding.class));
    }
    alg.setParameterSetIdentifier(table.getString("parameter_set_identifier"));
    alg.setCurve(table.getString("curve"));
    if (table.contains("execution_environment"))
      alg.setExecutionEnvironment(
        parseEnum(table, "execution_environment", ExecutionEnvironment.class));
    if (table.contains("implementation_platform"))
      alg.setImplementationPlatform(
        parseEnum(table, "implementation_platform", ImplementationPlatform.class));
    if (table.contains("crypto_functions"))
      alg.setCryptoFunctions(
        toEnumList(Objects.requireNonNull(table.getArray("crypto_functions")),
        CryptoFunction.class));
    if (table.contains("certification_level"))
      alg.setCertificationLevel(
        toEnumList(Objects.requireNonNull(table.getArray("certification_level")),
        CertificationLevel.class));
    if (table.contains("classical_security_level"))
      alg.setClassicalSecurityLevel(
        Math.toIntExact(table.getLong("classical_security_level")));
    if (table.contains("nist_quantum_security_level"))
      alg.setNistQuantumSecurityLevel(
        Math.toIntExact(table.getLong("nist_quantum_security_level")));
    props.setAlgorithmProperties(alg);
  }
  case CERTIFICATE -> {
    CertificateProperties cert = new CertificateProperties();
    cert.setSubjectName(table.getString("subject_name"));
    cert.setIssuerName(table.getString("issuer_name"));
    cert.setNotValidBefore(table.getString("not_valid_before"));
    cert.setNotValidAfter(table.getString("not_valid_after"));
    cert.setSignatureAlgorithmRef(table.getString("signature_algorithm_ref"));
    cert.setSubjectPublicKeyRef(table.getString("subject_public_key_ref"));
    cert.setCertificateFormat(table.getString("certificate_format"));
    cert.setCertificateExtension(table.getString("certificate_extension"));
    props.setCertificateProperties(cert);
  }
  case PROTOCOL -> {
    ProtocolProperties proto = new ProtocolProperties();
    if (table.contains("type"))
      proto.setType(parseEnum(table, "type", ProtocolType.class));
    proto.setVersion(table.getString("version"));
    if (table.contains("cipher_suites"))
      proto.setCipherSuites(
        parseCipherSuites(Objects.requireNonNull(table.getArray("cipher_suites"))));
    props.setProtocolProperties(proto);
  }
  case RELATED_CRYPTO_MATERIAL -> {
    RelatedCryptoMaterialProperties mat = new RelatedCryptoMaterialProperties();
    mat.setId(table.getString("id"));
    if (table.contains("state"))
      mat.setState(parseEnum(table, "state", State.class));
    mat.setAlgorithmRef(table.getString("algorithm_ref"));
    mat.setCreationDate(table.getString("creation_date"));
    mat.setActivationDate(table.getString("activation_date"));
    mat.setUpdateDate(table.getString("update_date"));
    mat.setExpirationDate(table.getString("expiration_date"));
```

```
          mat.setValue(table.getString("value"));
          if (table.contains("size"))
            mat.setSize(Math.toIntExact(table.getLong("size")));
          mat.setFormat(table.getString("format"));
          props.setRelatedCryptoMaterialProperties(mat);
        }
      }
      rule.setCryptoProperties(props);
      rules.add(rule);
    }
    policy.setRules(rules);
    return policy;
  }

  private static String requireString(TomlTable table, String key) {
    String value = table.getString(key);
    if (value == null)
      throw new IllegalArgumentException("Missing required string field: " + key);
    return value;
  }

  private static Long requireLong(TomlTable table, String key) {
    Long value = table.getLong(key);
    if (value == null)
      throw new IllegalArgumentException("Missing required long field: " + key);
    return value;
  }

  private static int requireInt(TomlTable table, String key) {
    return Math.toIntExact(requireLong(table, key));
  }

  private static TomlArray requireArray(TomlParseResult table, String key) {
    TomlArray array = table.getArray(key);
    if (array == null)
      throw new IllegalArgumentException("Missing required array field: " + key);
    return array;
  }

  private static <T extends Enum<T>> T parseEnum(TomlTable table, String key, Class<T> enumType) {
    String value = table.getString(key);
    if (value == null) throw new IllegalArgumentException("Missing enum field: " + key);
    try {
      return Enum.valueOf(enumType, value.toUpperCase());
    } catch (IllegalArgumentException e) {
      throw new IllegalArgumentException("Invalid value for enum " + key + ": " + value);
    }
  }

  private static <T extends Enum<T>> List<T> toEnumList(TomlArray array, Class<T> enumType) {
    return array.toList().stream()
      .map(v -> Enum.valueOf(enumType, v.toString().toUpperCase()))
      .collect(Collectors.toList());
  }

  private static List<CipherSuite> parseCipherSuites(TomlArray array) {
    return array.toList().stream()
      .map(s -> {
        CipherSuite cs = new CipherSuite();
        cs.setName(s.toString());
        return cs;
      })
      .collect(Collectors.toList());
  }
}
```

A.3 Custom Compliance Service

Listing A.3: Shortened implementation of `CustomComplianceService`

```
public class CustomComplianceService implements IComplianceService {
  private final CustomCompliancePolicy policy;
  private final Map<String, ComplianceLevel> levelMap;
  private final List<ComplianceLevel> levels;

  public CustomComplianceService(CustomCompliancePolicy policy) {
    this.policy = policy;
    this.levelMap = policy.getLevels().stream()
      .collect(Collectors.toMap(l -> String.valueOf(l.id()), l -> l));
    this.levels = new ArrayList<>(levelMap.values());
  }

  @Nonnull
  @Override
  public String getName() {
    return policy.getName();
  }

  @Nonnull
  @Override
  public List<ComplianceLevel> getComplianceLevels() {
    return new ArrayList<>(levels);
  }

  @Nonnull
  @Override
  public AssessmentLevel getDefaultAssessmentLevel() {
    return AssessmentLevel.UNKNOWN;
  }

  @Nonnull
  @Override
  public ComplianceLevel getDefaultComplianceLevel() {
    return levelMap.getOrDefault(
      String.valueOf(policy.getDefaultLevel()),
      levels.stream()
        .findFirst()
        .orElseThrow(() ->
          new IllegalStateException("No default compliance level defined"))
    );
  }

  @Nonnull
  @Override
  public ComplianceCheckResultDTO evaluate(
      @Nonnull PolicyIdentifier policyIdentifier,
      @Nonnull Collection<CryptographicAsset> assets) {

    List<ICryptographicAssetPolicyResult> results = new ArrayList<>();
    AssessmentLevel worstAssessment = getDefaultAssessmentLevel();

    for (CryptographicAsset asset : assets) {
      ICryptographicAssetPolicyResult result = evaluate(asset);
      results.add(result);

      ComplianceLevel level = result.complianceLevel();
      AssessmentLevel assessmentLevel = policy.getAssessmentLevels().stream()
        .filter(s -> s.getId() == level.assessmentId())
        .findFirst()
        .orElse(null);

      if (assessmentLevel != null &&
```

```
        assessmentLevel.getId() > worstAssessment.getId()) {
      worstAssessment = assessmentLevel;
    }
  }

  return new ComplianceCheckResultDTO(results, false, worstAssessment);
}

@Nonnull
private ICryptographicAssetPolicyResult evaluate(@Nonnull CryptographicAsset asset) {
  var props = asset.component().getCryptoProperties();

  int highestSpecificity = -1;
  ComplianceLevel bestLevel = getDefaultComplianceLevel();
  StringBuilder combinedDescription = new StringBuilder();
  String bestDescription = null;

  for (RuleDefinition rule : policy.getRules()) {
    CryptoProperties ruleProps = rule.getCryptoProperties();
    if (!Objects.equals(ruleProps.getAssetType(), props.getAssetType())) continue;

    String assetOid = props.getOid();
    String ruleOid = ruleProps.getOid();
    if (ruleOid != null && !matches(ruleOid, assetOid)) continue;

    String assetName = asset.component().getName();
    String ruleName = rule.getName();
    if (ruleName != null && !matches(ruleName, assetName)) continue;

    boolean matched = switch (props.getAssetType()) {
      case ALGORITHM ->
        matchAlgorithm(props.getAlgorithmProperties(), ruleProps.getAlgorithmProperties());
      case CERTIFICATE ->
        matchCertificate(props.getCertificateProperties(), ruleProps.getCertificateProperties());
      case PROTOCOL ->
        matchProtocol(props.getProtocolProperties(), ruleProps.getProtocolProperties());
      case RELATED_CRYPTO_MATERIAL ->
        matchRelatedMaterial(
          props.getRelatedCryptoMaterialProperties(),
          ruleProps.getRelatedCryptoMaterialProperties()
        );
      default -> false;
    };

    if (matched) {
      int specificity = computeSpecificity(rule);
      ComplianceLevel level = levelMap.getOrDefault(
        String.valueOf(rule.getLevelId()), getDefaultComplianceLevel());

      if (specificity > highestSpecificity ||
          (specificity == highestSpecificity && level.id() > bestLevel.id())) {
        highestSpecificity = specificity;
        bestLevel = level;
        bestDescription = rule.getDescription();
      }
    }
  }

  if (bestDescription != null) {
    combinedDescription.append("- ").append(bestDescription);
  } else {
    combinedDescription
      .append("No compliance rules matched for asset type: ")
      .append(props.getAssetType());
  }
```

```
  return new BasicCryptographicAssetPolicyResult(
    asset.identifier().toLowerCase(),
    bestLevel,
    combinedDescription.toString().trim()
  );
}

private boolean matchAlgorithm(AlgorithmProperties actual, AlgorithmProperties rule) {
  if (rule == null || actual == null) return false;
  if (!matches(rule.getPrimitive(), actual.getPrimitive())) return false;
  if (!matches(rule.getParameterSetIdentifier(), actual.getParameterSetIdentifier())) return
  false;
  if (!matches(rule.getCurve(), actual.getCurve())) return false;
  if (!matches(rule.getExecutionEnvironment(), actual.getExecutionEnvironment())) return false;
  if (!matches(rule.getImplementationPlatform(), actual.getImplementationPlatform())) return
  false;
  if (!matches(rule.getMode(), actual.getMode())) return false;
  if (!matches(rule.getPadding(), actual.getPadding())) return false;
  if (!matches(rule.getClassicalSecurityLevel(), actual.getClassicalSecurityLevel())) return
  false;
  if (!matches(rule.getNistQuantumSecurityLevel(), actual.getNistQuantumSecurityLevel())) return
  false;
  if (!matches(rule.getCertificationLevel(), actual.getCertificationLevel())) return false;
  if (!matches(rule.getCryptoFunctions(), actual.getCryptoFunctions())) return false;
  return true;
}

private boolean matchCertificate(CertificateProperties actual, CertificateProperties rule) {
  if (rule == null || actual == null) return false;
  if (!matches(rule.getSubjectName(), actual.getSubjectName())) return false;
  if (!matches(rule.getIssuerName(), actual.getIssuerName())) return false;
  if (!matches(rule.getNotValidBefore(), actual.getNotValidBefore())) return false;
  if (!matches(rule.getNotValidAfter(), actual.getNotValidAfter())) return false;
  if (!matches(rule.getSignatureAlgorithmRef(), actual.getSignatureAlgorithmRef())) return false;
  if (!matches(rule.getSubjectPublicKeyRef(), actual.getSubjectPublicKeyRef())) return false;
  if (!matches(rule.getCertificateFormat(), actual.getCertificateFormat())) return false;
  if (!matches(rule.getCertificateExtension(), actual.getCertificateExtension())) return false;
  return true;
}

private boolean matchProtocol(ProtocolProperties actual, ProtocolProperties rule) {
  if (rule == null || actual == null) return false;
  boolean result = true;
  result &= matches(rule.getType(), actual.getType());
  result &= matches(rule.getVersion(), actual.getVersion());
  if (rule.getCipherSuites() != null) {
    boolean suiteMatch = actual.getCipherSuites() != null &&
      actual.getCipherSuites().stream()
        .anyMatch(protoSuite -> rule.getCipherSuites().stream()
          .anyMatch(ruleSuite ->
            ruleSuite.getName().equalsIgnoreCase(protoSuite.getName())));
    result &= suiteMatch;
  }
  return result;
}

private boolean matchRelatedMaterial(
    RelatedCryptoMaterialProperties actual,
    RelatedCryptoMaterialProperties rule) {
  if (rule == null || actual == null) return false;
  if (!matches(rule.getType(), actual.getType())) return false;
  if (!matches(rule.getId(), actual.getId())) return false;
  if (!matches(rule.getState(), actual.getState())) return false;
  if (!matches(rule.getAlgorithmRef(), actual.getAlgorithmRef())) return false;
  if (!matches(rule.getCreationDate(), actual.getCreationDate())) return false;
  if (!matches(rule.getActivationDate(), actual.getActivationDate())) return false;
```

```
  if (!matches(rule.getUpdateDate(), actual.getUpdateDate())) return false;
  if (!matches(rule.getExpirationDate(), actual.getExpirationDate())) return false;
  if (!matches(rule.getValue(), actual.getValue())) return false;
  if (!matches(rule.getSize(), actual.getSize())) return false;
  if (!matches(rule.getFormat(), actual.getFormat())) return false;
  return true;
}

private ComplianceLevel updateWorstLevel(
    ComplianceLevel currentLevel,
    int newLevelId,
    String description,
    StringBuilder descriptionBuilder) {
  ComplianceLevel level = levelMap.getOrDefault(
    String.valueOf(newLevelId), getDefaultComplianceLevel());
  if (level.id() > currentLevel.id()) {
    currentLevel = level;
  }
  descriptionBuilder.append("- ").append(description).append("\n");
  return currentLevel;
}

private boolean matches(Object ruleValue, Object actualValue) {
  if (ruleValue == null) return true;
  if (actualValue == null) return false;

  if (ruleValue instanceof String ruleStr &&
      actualValue instanceof String actualStr) {
    return ruleStr.trim().equalsIgnoreCase(actualStr.trim());
  }
  if (ruleValue instanceof Enum<?> ruleEnum &&
      actualValue instanceof Enum<?> actualEnum) {
    return ruleEnum.name().equalsIgnoreCase(actualEnum.name());
  }
  if (ruleValue instanceof Integer ruleInt &&
      actualValue instanceof Integer actualInt) {
    return ruleInt.equals(actualInt);
  }
  if (ruleValue instanceof Number ruleNum &&
      actualValue instanceof Number actualNum) {
    return ruleNum.intValue() == actualNum.intValue();
  }
  return ruleValue.toString().trim()
    .equalsIgnoreCase(actualValue.toString().trim());
}

private int computeSpecificity(@Nonnull RuleDefinition rule) {
  int specificity = 0;
  if (rule.getName() != null && !rule.getName().isEmpty()) specificity++;

  CryptoProperties props = rule.getCryptoProperties();
  if (props != null) {
    if (props.getAssetType() != null) specificity++;

    AlgorithmProperties algo = props.getAlgorithmProperties();
    if (algo != null) {
      if (algo.getPrimitive() != null) specificity++;
      if (algo.getCryptoFunctions() != null && !algo.getCryptoFunctions().isEmpty())
  specificity++;
      if (algo.getCurve() != null) specificity++;
      if (algo.getMode() != null) specificity++;
      if (algo.getPadding() != null) specificity++;
      if (algo.getNistQuantumSecurityLevel() != null) specificity++;
      if (algo.getParameterSetIdentifier() != null) specificity++;
      if (algo.getExecutionEnvironment() != null) specificity++;
      if (algo.getImplementationPlatform() != null) specificity++;
```

```
        if (algo.getCertificationLevel() != null) specificity++;
        if (algo.getClassicalSecurityLevel() != null) specificity++;
      }

      CertificateProperties cert = props.getCertificateProperties();
      if (cert != null) {
        if (cert.getSubjectName() != null) specificity++;
        if (cert.getIssuerName() != null) specificity++;
        if (cert.getNotValidBefore() != null) specificity++;
        if (cert.getNotValidAfter() != null) specificity++;
        if (cert.getSignatureAlgorithmRef() != null) specificity++;
        if (cert.getSubjectPublicKeyRef() != null) specificity++;
        if (cert.getCertificateFormat() != null) specificity++;
        if (cert.getCertificateExtension() != null) specificity++;
      }

      ProtocolProperties proto = props.getProtocolProperties();
      if (proto != null) {
        if (proto.getType() != null) specificity++;
        if (proto.getVersion() != null) specificity++;
        if (proto.getCipherSuites() != null && !proto.getCipherSuites().isEmpty()) specificity++;
      }

      RelatedCryptoMaterialProperties mat = props.getRelatedCryptoMaterialProperties();
      if (mat != null) {
        if (mat.getType() != null) specificity++;
        if (mat.getId() != null) specificity++;
        if (mat.getState() != null) specificity++;
        if (mat.getAlgorithmRef() != null) specificity++;
        if (mat.getCreationDate() != null) specificity++;
        if (mat.getActivationDate() != null) specificity++;
        if (mat.getUpdateDate() != null) specificity++;
        if (mat.getExpirationDate() != null) specificity++;
        if (mat.getValue() != null) specificity++;
        if (mat.getSize() != null) specificity++;
        if (mat.getFormat() != null) specificity++;
      }
    }
    return specificity;
  }
}
```

Appendix B

Policy and CBOM Examples

This appendix contains the full TOML policy rule definitions and CBOM JSON excerpts referenced in Chapter 4. Only shortened excerpts were shown in the main text; complete examples are provided here for reference.

B.1 Policy Rules (TOML)

Listing B.1: Excerpt of TOML ruleset used in the test CBOM evaluation

```
[[rule]]
mode = "ecb"
compliance_level = 3
description = "ECB mode is disallowed for encryption, allowed for decryption (Section 2.2)"
asset_type = "ALGORITHM"

[[rule]]
mode = "ecb"
compliance_level = 2
description = "ECB mode is allowed for decryption (Section 2.2)"
crypto_functions = ["DECRYPT"]
asset_type = "ALGORITHM"

[[rule]]
mode = "ecb"
compliance_level = 5
description = "ECB mode is disallowed for encryption (Section 2.2)"
crypto_functions = ["ENCRYPT"]
asset_type = "ALGORITHM"

[[rule]]
mode = "ecb"
compliance_level = 5
description = "ECB mode is disallowed for encryption (Section 2.2)"
crypto_functions = ["DECRYPT", "ENCRYPT"]
asset_type = "ALGORITHM"

[[rule]]
name = "sha1"
compliance_level = 4
description = "SHA-1 is deprecated and disallowed after 2030 (Section 1.2)"
asset_type = "ALGORITHM"

[[rule]]
name = "3des"
compliance_level = 5
description = "TDEA (3DES) encryption is disallowed (Section 2.1)"
asset_type = "ALGORITHM"
```

B.2 Test CBOM Example (JSON)

Listing B.2: Test CBOM used for engine validation


```
{
  "components": [
    {
      "type": "cryptographic-asset",
      "bom-ref": "01cd7b57-69d1-4bbd-a710-dd98cdb26296",
      "name": "AES128-ECB-PKCS5",
      "cryptoProperties": {
        "assetType": "algorithm",
        "algorithmProperties": {
          "primitive": "block-cipher",
          "parameterSetIdentifier": "128",
          "mode": "ecb",
          "padding": "pkcs5",
          "cryptoFunctions": ["decrypt", "encrypt"]
        },
        "oid": "2.16.840.1.101.3.4.1"
      }
    },
    {
      "type": "cryptographic-asset",
      "bom-ref": "34ec56ce-85b0-4694-a446-5b988c9f0e3a",
      "name": "SHA1",
      "cryptoProperties": {
        "assetType": "algorithm",
        "algorithmProperties": {
          "primitive": "hash",
          "parameterSetIdentifier": "160",
          "cryptoFunctions": ["digest"]
        },
        "oid": "1.3.14.3.2.26"
      }
    },
    {
      "type": "cryptographic-asset",
      "bom-ref": "afb96b63-8fae-49f6-af56-1224fc9e7bf3",
      "name": "3DES",
      "cryptoProperties": {
        "assetType": "algorithm",
        "algorithmProperties": {
          "primitive": "block-cipher",
          "parameterSetIdentifier": "168",
          "cryptoFunctions": ["keygen"]
        }
      }
    }
  ]
}
```

B.3 Keycloak CBOM Excerpt (JSON)

Listing B.3: Keycloak CBOM excerpt showing SHA-1 usage


```
{
  "type": "cryptographic-asset",
  "bom-ref": "c6a76fd5-c091-4553-a157-79850036cfec",
  "name": "SHA1",
  "evidence": {
    "occurrences": [
      {
        "location": "server-spi-private/src/main/java/
        org/keycloak/protocol/saml/util/ArtifactBindingUtils.java",
        "line": 45,
        "additionalContext":
          "java.security.MessageDigest#getInstance(Ljava/lang/String;)Ljava/
          security/MessageDigest;"
      },
      {
        "location": "federation/sssd/src/main/java/
        org/freedesktop/dbus/connections/SASL.java",
        "line": 334,
        "additionalContext":
          "java.security.MessageDigest#getInstance(Ljava/lang/String;)Ljava/
          security/MessageDigest;"
      }
    ]
  }
}
```